

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế xây dựng hệ thống

NGUYỄN ĐỨC THẨM

tham.nd225395@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: TS. Lê Bá Vui

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 07/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Thiết kế xây dựng hệ thống giám sát chất lượng
không khí sử dụng mạng cảm biến không dây LoRa**

NGUYỄN ĐỨC THẮM

tham.nd225395@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: PGS. TS. Lê Bá Vui

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 07/2026

LỜI CẢM ƠN

Lời cảm ơn của sinh viên (SV) tới người yêu, gia đình, bạn bè, thầy cô, và chính bản thân mình vì đã chăm chỉ và quyết tâm thực hiện ĐATN để đạt kết quả tốt nhất, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng, giới hạn trong khoảng 100-150 từ.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sinh viên viết tóm tắt ĐATN của mình trong mục này, với 200 đến 350 từ. Theo trình tự, các nội dung tóm tắt cần có: (i) Giới thiệu vấn đề (tại sao có vấn đề đó, hiện tại được giải quyết chưa, có những hướng tiếp cận nào, các hướng này giải quyết như thế nào, hạn chế là gì), (ii) Hướng tiếp cận sinh viên lựa chọn là gì, vì sao chọn hướng đó, (iii) Tổng quan giải pháp của sinh viên theo hướng tiếp cận đã chọn, và (iv) Đóng góp chính của ĐATN là gì, kết quả đạt được sau cùng là gì. Sinh viên cần viết thành đoạn văn, không được viết ý hoặc gạch đầu dòng.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đề án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.1.1 PAM Air	4
2.1.2 IQAir AirVisual	4
2.1.3 PurpleAir	5
2.1.4 Hệ thống quan trắc quốc gia VN-AQI	5
2.1.5 So sánh và đánh giá tổng hợp	5
2.2 Tổng quan chức năng	6
2.2.1 Biểu đồ use case tổng quát	6
2.2.2 Biểu đồ use case phân rã nhóm chức năng Người dùng	7
2.2.3 Biểu đồ use case phân rã nhóm chức năng Quản trị	9
2.2.4 Quy trình nghiệp vụ	10
2.3 Đặc tả chức năng	12
2.3.1 Đặc tả use case Xem dashboard chất lượng không khí	13
2.3.2 Đặc tả use case Xem lịch sử dữ liệu	14
2.3.3 Đặc tả use case Quản lý thiết bị	14
2.3.4 Đặc tả use case Giám sát và xử lý cảnh báo	15
2.3.5 Đặc tả use case Xuất dữ liệu CSV	15
2.4 Yêu cầu phi chức năng	16

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	17
3.1 Kiến trúc tổng thể hệ thống	17
3.2 Nền tảng phần cứng IoT	18
3.2.1 Vi điều khiển ESP32	18
3.2.2 Module cảm biến	19
3.3 Giao thức truyền thông LoRa.....	20
3.3.1 Tổng quan về LoRa	20
3.3.2 Module AS32-TTL-100	20
3.4 Công nghệ phần mềm phía máy chủ	21
3.4.1 Node.js và Express	21
3.4.2 PostgreSQL, TimescaleDB và PostGIS	21
3.4.3 Prisma ORM	22
3.4.4 Socket.IO	22
3.5 Công nghệ phần mềm phía giao diện	23
3.5.1 React và Vite	23
3.5.2 Thư viện trực quan hóa dữ liệu	23
3.6 Lý thuyết tính toán chỉ số chất lượng không khí	23
3.6.1 Khái niệm và phân loại mức AQI.....	23
3.6.2 Công thức và bảng điểm ngắt	24
3.6.3 Ví dụ tính toán.....	25
3.7 Hạ tầng triển khai	25
3.7.1 Docker và Docker Compose	25
3.7.2 Nginx.....	26

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	27
4.1 Thiết kế kiến trúc.....	27
4.1.1 Lựa chọn kiến trúc phần mềm	27
4.1.2 Thiết kế tổng quan.....	28
4.1.3 Mô tả luồng dữ liệu giữa các gói.....	29
4.2 Thiết kế chi tiết.....	30
4.2.1 Thiết kế phần cứng.....	30
4.2.2 Thiết kế firmware.....	31
4.2.3 Thiết kế giao diện	32
4.2.4 Thiết kế module Backend.....	33
4.2.5 Thiết kế cơ sở dữ liệu	36
4.3 Xây dựng ứng dụng.....	38
4.3.1 Thư viện và công cụ sử dụng.....	38
4.3.2 Kết quả đạt được	39
4.3.3 Minh họa các chức năng chính	40
4.4 Kiểm thử.....	44
4.4.1 Kiểm thử chức năng Xem dashboard.....	45
4.4.2 Kiểm thử chức năng Quản lý thiết bị.....	45
4.4.3 Kiểm thử chức năng Giám sát cảnh báo.....	46
4.4.4 Kiểm thử đơn vị module tính AQI.....	46
4.4.5 Tổng kết kiểm thử	47
4.5 Triển khai	48
4.5.1 Kết quả vận hành thực tế.....	49
4.5.2 So sánh với các hệ thống hiện có	50
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	52

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	53
6.1 Kết luận.....	53
6.2 Hướng phát triển.....	53
TÀI LIỆU THAM KHẢO.....	57
PHỤ LỤC.....	59
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	59
A.1 Ngành học.....	60
A.2 Đánh dấu (bullet) và đánh số (numering)	60
A.3 Cách thêm bảng	61
A.4 Chèn hình ảnh	61
A.5 Tài liệu tham khảo	62
A.6 Cách viết phương trình và công thức toán học.....	62
A.7 Qui cách đóng quyển.....	62
B. ĐẶC TẢ USE CASE.....	64
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	64
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	64

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát của hệ thống	6
Hình 2.2	Biểu đồ use case phân rã nhóm chức năng Người dùng	7
Hình 2.3	Biểu đồ use case phân rã nhóm chức năng Quản trị	9
Hình 2.4	Biểu đồ hoạt động quy trình thu thập và hiển thị dữ liệu	11
Hình 2.5	Biểu đồ hoạt động quy trình phát hiện và xử lý cảnh báo	12
Hình 3.1	Kiến trúc tổng thể hệ thống giám sát chất lượng không khí	18
Hình 4.1	Biểu đồ gói UML tổng quan của hệ thống	28
Hình 4.2	Biểu đồ quan hệ module cho backend services	29
Hình 4.3	Sơ đồ mạch Sensor Node	30
Hình 4.4	Sơ đồ mạch Gateway	31
Hình 4.5	Biểu đồ trình tự xử lý dữ liệu Telemetry	34
Hình 4.6	Biểu đồ trình tự xem Dashboard thời gian thực	35
Hình 4.7	Biểu đồ trình tự xử lý cảnh báo vượt ngưỡng	36
Hình 4.8	Entity-Relationship Diagram	37
Hình 4.9	Dashboard chất lượng không khí	41
Hình 4.10	Lịch sử dữ liệu	42
Hình 4.11	Bản đồ AQI	42
Hình 4.12	Xếp hạng ô nhiễm	43
Hình 4.13	Xuất dữ liệu CSV	43
Hình 4.14	Quản lý tài khoản	44
Hình 4.15	Nhật ký hệ thống	44
Hình 4.16	Biểu đồ triển khai hệ thống	48
Hình 4.17	Kết quả lệnh <code>docker ps</code> trên máy chủ production	49
Hình 4.18	Biểu đồ dữ liệu PM2.5 và AQI thực tế thu thập qua 48 giờ	50
Hình A.1	Internet vạn vật	61
Hình A.2	Quy cách đóng quyển đồ án	63
Hình A.3	Quy cách đóng quyển đồ án	63

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các hệ thống giám sát chất lượng không khí hiện có . . .	5
Bảng 2.2	Đặc tả use case Xem dashboard chất lượng không khí	13
Bảng 2.3	Đặc tả use case Xem lịch sử dữ liệu	14
Bảng 2.4	Đặc tả use case Quản lý thiết bị	14
Bảng 2.5	Đặc tả use case Giám sát và xử lý cảnh báo	15
Bảng 2.6	Đặc tả use case Xuất dữ liệu CSV	15
Bảng 2.7	Các yêu cầu phi chức năng của hệ thống	16
Bảng 3.1	Thông số kỹ thuật các cảm biến sử dụng trong hệ thống	20
Bảng 3.2	Phân loại các mức chỉ số AQI theo tiêu chuẩn US EPA [8] . .	24
Bảng 3.3	Bảng điểm ngắt AQI cho PM2.5 [8]	24
Bảng 3.4	Bảng điểm ngắt AQI cho PM10 [8]	25
Bảng 4.1	Danh sách thư viện và công cụ phía Backend	38
Bảng 4.2	Danh sách thư viện và công cụ phía Frontend	38
Bảng 4.3	Thống kê quy mô mã nguồn hệ thống	39
Bảng 4.4	Đối sánh yêu cầu đặc tả với kết quả đạt được	40
Bảng 4.5	Trường hợp kiểm thử chức năng Xem dashboard	45
Bảng 4.6	Trường hợp kiểm thử chức năng Quản lý thiết bị	45
Bảng 4.7	Trường hợp kiểm thử chức năng Giám sát cảnh báo	46
Bảng 4.8	Kết quả kiểm thử đơn vị module <code>aqiService</code>	47
Bảng 4.9	So sánh hệ thống đề xuất với các hệ thống hiện có	50
Bảng A.1	Table to test captions and labels.	61

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
ACID	Tính nguyên tử, nhất quán, cô lập, bền vững (Atomicity, Consistency, Isolation, Durability)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
AQI	Chỉ số chất lượng không khí (Air Quality Index)
CRUD	Tạo, Đọc, Cập nhật, Xóa (Create, Read, Update, Delete)
CSV	Giá trị phân cách bằng dấu phẩy (Comma-Separated Values)
GPS	Hệ thống định vị toàn cầu (Global Positioning System)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
HTTP	Giao thức truyền siêu văn bản (HyperText Transfer Protocol)
IoT	Internet vạn vật (Internet of Things)
JSON	Ký pháp đối tượng JavaScript (JavaScript Object Notation)
JWT	Mã thông báo web JSON (JSON Web Token)
LoRa	Truyền thông tầm xa công suất thấp (Long Range)
MCU	Vi điều khiển (Microcontroller Unit)
ORM	Ánh xạ đối tượng – quan hệ (Object-Relational Mapping)
REST	Chuyển giao trạng thái đại diện (Representational State Transfer)
RTOS	Hệ điều hành thời gian thực (Real-Time Operating System)
SPA	Ứng dụng trang đơn (Single-Page Application)

Thuật ngữ	Ý nghĩa
TVOC	Tổng các hợp chất hữu cơ bay hơi (Total Volatile Organic Compounds)
UART	Bộ thu phát không đồng bộ đa năng (Universal Asynchronous Receiver-Transmitter)
US EPA	Cơ quan Bảo vệ Môi trường Hoa Kỳ (United States Environmental Protection Agency)
WHO	Tổ chức Y tế Thế giới (World Health Organization)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Lưu ý: **Trước khi viết ĐATN, sinh viên cần đọc kỹ hướng dẫn và quy định chi tiết** về cách viết ĐATN trong Phụ lục A. Sinh viên tuân theo mẫu tài liệu này để viết báo cáo đồ án tốt nghiệp, vì tài liệu này đã được căn chỉnh, chỉnh sửa theo đúng chuẩn báo cáo kỹ thuật đồ án tốt nghiệp (ISO 7144:1986). Sinh viên viết trực tiếp vào file này, chỉ chỉnh sửa nội dung, và không viết trên file mới.

Khi đóng quyển ĐATN, sinh viên cần lưu ý tuân thủ hướng dẫn ở phụ lục A.9

SV cần đặc biệt lưu ý cách hành văn. Mỗi đoạn văn không được quá dài và cần có ý tứ rõ ràng, bao gồm duy nhất một ý chính và các ý phân tích bổ trợ để làm rõ hơn ý chính. Các câu văn trong đoạn phải đầy đủ chủ ngữ vị ngữ, cùng hướng đến chủ đề chung. Câu sau phải liên kết với câu trước, đoạn sau liên kết với đoạn trước. Trong văn phong khoa học, sinh viên không được dùng từ trong văn nói, không dùng các từ phóng đại, thái quá, các từ thiếu khách quan, thiên về cảm xúc, về quan điểm cá nhân như “tuyệt vời”, “cực hay”, “cực kỳ hữu ích”, v.v. Các câu văn cần được tối ưu hóa, đảm bảo rất khó để thể thêm hoặc bớt đi được dù chỉ một từ. Cách diễn đạt cần ngắn gọn, súc tích, không dài dòng.

Mẫu ĐATN này được thiết kế phù hợp nhất với đa số các đề tài xây dựng phần mềm ứng dụng. Với các dạng đề tài khác (giải pháp, nghiên cứu, phần mềm đặc thù, v.v.), sinh viên dựa trên cấu trúc và hướng dẫn của báo cáo này để đề xuất và trao đổi với giáo viên hướng dẫn để thiết kế khung báo cáo đồ án cho phù hợp. Sinh viên lưu ý **trong mọi trường hợp, SV luôn phải sử dụng định dạng báo cáo này, và phải đọc kỹ toàn bộ các hướng dẫn từ đầu tới cuối.** Các hướng dẫn không chỉ áp dụng riêng cho đề tài ứng dụng, mà còn phù hợp với các dạng đề tài khác. Ngoài ra, trong mẫu ĐATN này đã được tích hợp một số hướng dẫn dành riêng cho đề tài nghiên cứu.

Chương 1 có độ dài từ 3 đến 6 trang với các nội dung sau đây

1.1 Đặt vấn đề

Khi đặt vấn đề, sinh viên cần làm nổi bật mức độ cấp thiết, tầm quan trọng và/hoặc quy mô của bài toán của mình.

Gợi ý cách trình bày cho sinh viên: Xuất phát từ tình hình thực tế gì, dẫn đến vấn đề hoặc bài toán gì. Vấn đề hoặc bài toán đó, nếu được giải quyết, đem lại lợi ích gì, cho những ai, còn có thể được áp dụng vào các lĩnh vực khác nữa không. Sinh viên cần lưu ý phần này chỉ trình bày vấn đề, tuyệt đối không trình bày giải pháp.

1.2 Mục tiêu và phạm vi đề tài

Sinh viên trước tiên cần trình bày tổng quan các kết quả của các nghiên cứu hiện nay cho bài toán giới thiệu ở phần 1.1 (đối với đề tài nghiên cứu), hoặc về các sản phẩm hiện tại/về nhu cầu của người dùng (đối với đề tài ứng dụng). Tiếp đến, sinh viên tiến hành so sánh và đánh giá tổng quan các sản phẩm/nghiên cứu này.

Dựa trên các phân tích và đánh giá ở trên, sinh viên khái quát lại các hạn chế hiện tại đang gặp phải. Trên cơ sở đó, sinh viên sẽ hướng tới giải quyết vấn đề cụ thể gì, khắc phục hạn chế gì, phát triển phần mềm **có các chức năng chính gì**, tạo nên đột phá gì, v.v.

Trong phần này, sinh viên lưu ý chỉ trình bày tổng quan, không đi vào chi tiết của vấn đề hoặc giải pháp. Nội dung chi tiết sẽ được trình bày trong các chương tiếp theo, đặc biệt là trong Chương 5.

1.3 Định hướng giải pháp

Từ việc xác định rõ nhiệm vụ cần giải quyết ở phần 1.2, sinh viên đề xuất định hướng giải pháp của mình theo trình tự sau: (i) Sinh viên trước tiên trình bày sẽ giải quyết vấn đề theo định hướng, phương pháp, thuật toán, kỹ thuật, hay công nghệ nào; Tiếp theo, (ii) sinh viên mô tả ngắn gọn giải pháp của mình là gì (khi đi theo định hướng/phương pháp nêu trên); và sau cùng, (iii) sinh viên trình bày đóng góp chính của đề án là gì, kết quả đạt được là gì.

Sinh viên lưu ý không giải thích hoặc phân tích chi tiết công nghệ/thuật toán trong phần này. Sinh viên chỉ cần nêu tên định hướng công nghệ/thuật toán, mô tả ngắn gọn trong một đến hai câu và giải thích nhanh lý do lựa chọn.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày về v.v.

Trong Chương 3, em/tôi giới thiệu về v.v.

Chú ý: Sinh viên cần viết mô tả thành đoạn văn đầy đủ về nội dung chương. Tuyệt đối không viết ý hay gạch đầu dòng. Chương 1 không cần mô tả trong phần này.

Ví dụ tham khảo mô tả chương trong phần bố cục đề án tốt nghiệp: Chương *** trình bày đóng góp chính của đề án, đó là một nền tảng ABC cho phép khai phá và tích hợp nhiều nguồn dữ liệu, trong đó mỗi nguồn dữ liệu lại có định dạng đặc thù riêng. Nền tảng ABC được phát triển dựa trên khái niệm DEF, là các module ngữ nghĩa trợ giúp người dùng tìm kiếm, tích hợp và hiển thị trực quan dữ liệu theo mô

hình cộng tác và mô hình phân tán.

Chú ý: Trong phần nội dung chính, mỗi chương của đề án nên có phần Tổng quan và Kết chương. Hai phần này đều có định dạng văn bản “Normal”, sinh viên không cần tạo định dạng riêng, ví dụ như không in đậm/in nghiêng, không đóng khung, v.v.

Trong phần Tổng quan của chương N, sinh viên nên có sự liên kết với chương N-1 rồi trình bày sơ qua lý do có mặt của chương N và sự cần thiết của chương này trong đề án. Sau đó giới thiệu những vấn đề sẽ trình bày trong chương này là gì, trong các đề mục lớn nào.

Ví dụ về phần Tổng quan: Chương 3 đã thảo luận về nguồn gốc ra đời, cơ sở lý thuyết và các nhiệm vụ chính của bài toán tích hợp dữ liệu. Chương 4 này sẽ trình bày chi tiết các công cụ tích hợp dữ liệu theo hướng tiếp cận “mashup”. Với mục đích và phạm vi của đề tài, sáu nhóm công cụ tích hợp dữ liệu chính được trình bày bao gồm: (i) nhóm công cụ ABC trong phần 4.1, (ii) nhóm công cụ DEF trong phần 4.2, nhóm công cụ GHK trong phần 4.3, v.v.

Trong phần Kết chương, sinh viên đưa ra một số kết luận quan trọng của chương. Những vấn đề mở ra trong Tổng quan cần được tóm tắt lại nội dung và cách giải quyết/thực hiện như thế nào. Sinh viên lưu ý không viết Kết chương giống hệt Tổng quan. Sau khi đọc phần Kết chương, người đọc sẽ nắm được sơ bộ nội dung và giải pháp cho các vấn đề đã trình bày trong chương. Trong Kết chương, Sinh viên nên có thêm câu liên kết tới chương tiếp theo.

Ví dụ về phần Kết chương: Chương này đã phân tích chi tiết sáu nhóm công cụ tích hợp dữ liệu. Nhóm công cụ ABC và DEF thích hợp với những bài toán tích hợp dữ liệu phạm vi nhỏ. Trong khi đó, nhóm công cụ GHK lại chứng tỏ thế mạnh của mình với những bài toán cần độ chính xác cao, v.v. Từ kết quả nghiên cứu và phân tích về sáu nhóm công cụ tích hợp dữ liệu này, tôi đã thực hiện phát triển phần mềm tự động bóc tách và tích hợp dữ liệu sử dụng nhóm công cụ GHK. Phần này được trình bày trong chương tiếp theo – Chương 5.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã giới thiệu bối cảnh thực tiễn của bài toán giám sát chất lượng không khí đô thị, xác định mục tiêu và phạm vi đề tài, đồng thời đưa ra định hướng giải pháp dựa trên mạng cảm biến không dây LoRa. Trên cơ sở đó, Chương 2 tiến hành khảo sát các hệ thống giám sát chất lượng không khí hiện có, phân tích ưu nhược điểm và xác định khoảng trống cần giải quyết. Tiếp theo, chương trình bày phân tích yêu cầu chức năng thông qua biểu đồ use case, đặc tả chi tiết các chức năng chính, và các yêu cầu phi chức năng của hệ thống.

2.1 Khảo sát hiện trạng

Để xác định rõ hạn chế của các giải pháp hiện tại và định hướng phát triển hệ thống, em tiến hành khảo sát bốn hệ thống giám sát chất lượng không khí tiêu biểu, bao gồm cả hệ thống trong nước và quốc tế.

2.1.1 PAM Air

PAM Air là hệ thống giám sát chất lượng không khí do công ty Việt Nam PAM Air JSC phát triển, triển khai mạng lưới hơn 3.000 trạm đo trên toàn quốc [1]. Hệ thống sử dụng các cảm biến chi phí thấp, kết nối WiFi hoặc 4G, đo chủ yếu bụi mịn PM_{2.5} — một trong những chỉ số quan trọng nhất theo khuyến nghị của WHO [2]. Dữ liệu được hiển thị trên ứng dụng di động và trang web với bản đồ theo thời gian thực.

Tuy nhiên, PAM Air có một số hạn chế. Thứ nhất, phần cứng và phần mềm đều là mã nguồn đóng, không cho phép tùy biến hoặc mở rộng. Thứ hai, mỗi trạm đo yêu cầu kết nối WiFi hoặc 4G riêng, hạn chế khả năng triển khai tại các khu vực không có hạ tầng mạng. Thứ ba, hệ thống chủ yếu đo PM_{2.5}, không tích hợp đo CO₂ hoặc TVOC — những thông số mà Morawska và cộng sự [3] đã chỉ ra là cần thiết cho đánh giá toàn diện chất lượng không khí trong nhà và ngoài trời.

2.1.2 IQAir AirVisual

IQAir AirVisual là nền tảng quốc tế cung cấp dữ liệu chất lượng không khí từ hơn 100 quốc gia, kết hợp dữ liệu từ các trạm quan trắc chính thức và các thiết bị đo cộng đồng [4]. Hệ thống có ứng dụng di động và trang web với giao diện trực quan, hỗ trợ dự báo chất lượng không khí dựa trên mô hình AI.

Hạn chế chính của IQAir là chi phí thiết bị đo cá nhân cao (khoảng 269 USD cho AirVisual Pro). Dữ liệu phụ thuộc vào hạ tầng đám mây quốc tế, gây rủi ro về độ trễ và quyền kiểm soát dữ liệu. Ngoài ra, người dùng không thể tự triển khai hệ thống riêng vì phần mềm không phải mã nguồn mở.

2.1.3 PurpleAir

PurpleAir là hệ thống giám sát chất lượng không khí dựa trên mô hình cộng đồng, bán các trạm đo cá nhân sử dụng cảm biến laser (Plantower PMS5003) [5]. Dữ liệu từ tất cả các trạm được tổng hợp trên bản đồ trực tuyến công khai. PurpleAir cung cấp API mở cho phép truy xuất dữ liệu.

Tuy nhiên, PurpleAir chỉ đo bụi mịn PM2.5 và PM10, không hỗ trợ đo khí CO₂, TVOC, nhiệt độ hoặc độ ẩm. Mỗi trạm đo yêu cầu kết nối WiFi trực tiếp. Hệ thống không hỗ trợ giao thức truyền tầm xa như LoRa — một giao thức đã được chứng minh phù hợp cho các ứng dụng IoT tầm xa, tiêu thụ năng lượng thấp [6] — do đó không phù hợp để triển khai mạng lưới phân tán trên diện rộng mà không có hạ tầng WiFi.

2.1.4 Hệ thống quan trắc quốc gia VN-AQI

Hệ thống quan trắc chất lượng không khí quốc gia do Bộ Tài nguyên và Môi trường vận hành, sử dụng các trạm đo chuyên dụng tiêu chuẩn FEM (Federal Equivalent Method) [7]. Dữ liệu được công bố trên trang enviinfo.cem.gov.vn theo chỉ số VN-AQI. Ưu điểm nổi bật là độ chính xác cao, đáp ứng tiêu chuẩn quốc tế.

Hạn chế lớn nhất là chi phí triển khai cực cao, mỗi trạm có giá từ hàng trăm triệu đến hàng tỷ đồng. Số lượng trạm rất ít (khoảng 35 trạm tự động trên cả nước), không đủ phủ sóng mức vi mô cho từng khu vực đô thị.

2.1.5 So sánh và đánh giá tổng hợp

Bảng 2.1 tổng hợp so sánh bốn hệ thống đã khảo sát theo các tiêu chí quan trọng.

Bảng 2.1: So sánh các hệ thống giám sát chất lượng không khí hiện có

Tiêu chí	PAM Air	IQAir	PurpleAir	VN-AQI
Cảm biến	PM2.5	PM2.5, CO ₂	PM2.5, PM10	Đầy đủ (FEM)
Truyền thông	WiFi/4G	WiFi	WiFi	Chuyên dụng
Chi phí/trạm	Trung bình	Cao	Trung bình	Rất cao
Mã nguồn mở	Không	Không	Một phần	Không
Tùy biến	Không	Không	Hạn chế	Không
LoRa	Không	Không	Không	Không
Realtime web	Có	Có	Có	Hạn chế
Admin panel	Không rõ	Không	Không	Có (nội bộ)

Từ kết quả khảo sát, có thể nhận thấy các hệ thống hiện tại đều có những hạn chế nhất định. Các hệ thống thương mại như PAM Air và IQAir không cho phép tùy biến và mở rộng. Các hệ thống cộng đồng như PurpleAir chỉ đo được bụi mịn và

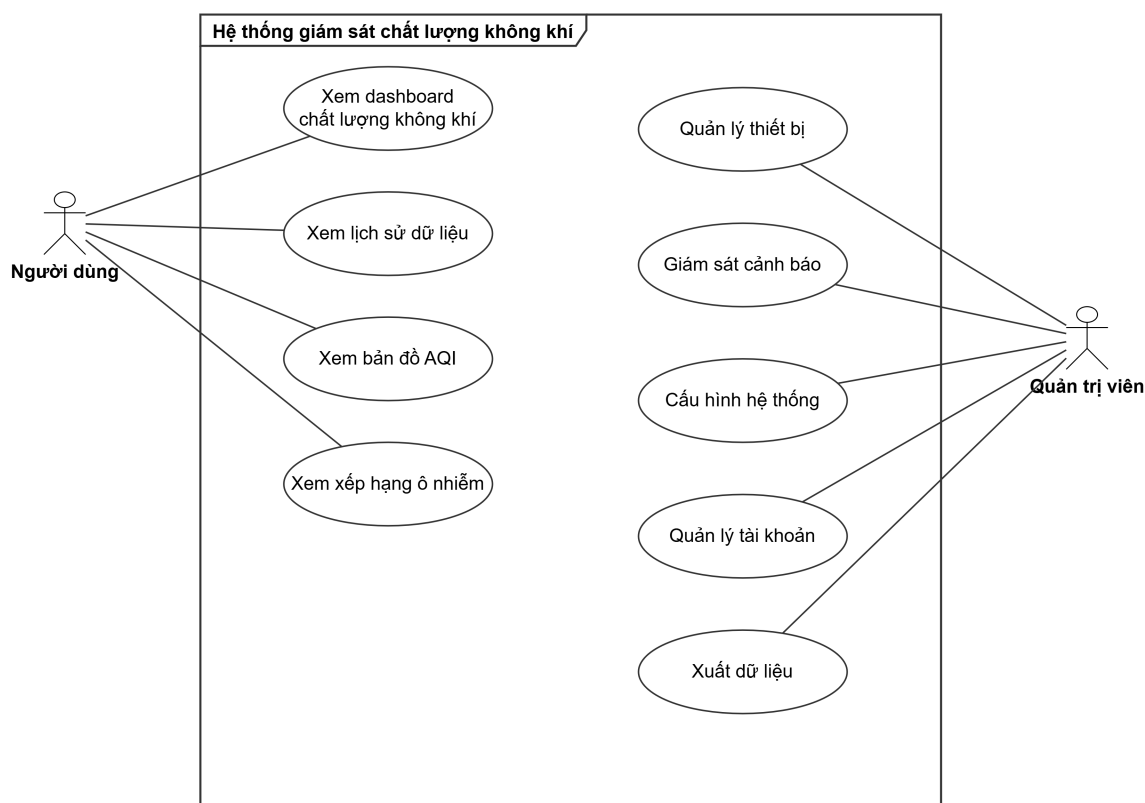
yêu cầu WiFi cho mỗi trạm. Hệ thống quan trắc quốc gia đáp ứng tiêu chuẩn FEM nhưng chi phí quá cao cho quy mô triển khai vi mô. Đặc biệt, không hệ thống nào hỗ trợ giao thức LoRa để truyền dữ liệu tầm xa mà không cần hạ tầng mạng WiFi tại mỗi điểm đo — trong khi LoRa đã được chứng minh có khả năng truyền thông đến 10–15 km trong điều kiện tầm nhìn thẳng với mức tiêu thụ năng lượng rất thấp [6].

Hệ thống đề xuất trong đề án này hướng tới giải quyết các hạn chế trên bằng cách kết hợp: (i) cảm biến chi phí thấp đo đa thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ, độ ẩm) theo khuyến nghị của Morawska và cộng sự [3], (ii) truyền thông LoRa không cần WiFi cho mỗi node [6], (iii) mã nguồn mở hoàn toàn, (iv) dashboard web thời gian thực với chỉ số AQI theo chuẩn US EPA [8], và (v) bảng quản trị cho quản trị viên.

2.2 Tổng quan chức năng

Phần này trình bày tổng quan các chức năng của hệ thống thông qua biểu đồ use case. Hệ thống có hai tác nhân chính: **Người dùng** (User) và **Quản trị viên** (Admin). Người dùng là đối tượng sử dụng dashboard web để tra cứu thông tin chất lượng không khí. Quản trị viên là người quản lý hệ thống, bao gồm quản lý thiết bị, cấu hình, theo dõi cảnh báo và xuất dữ liệu.

2.2.1 Biểu đồ use case tổng quát



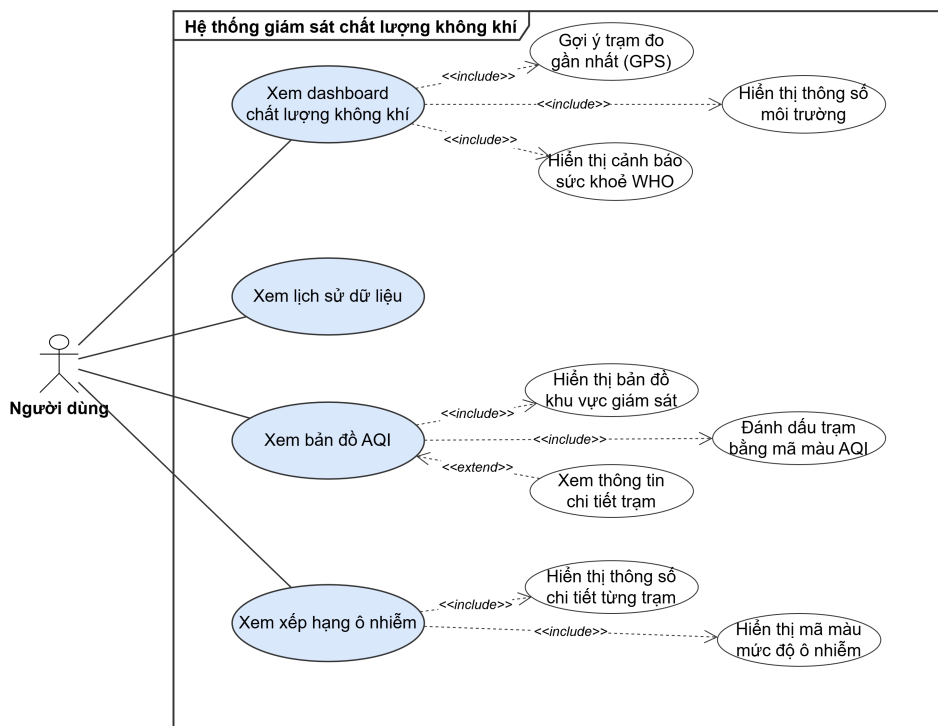
Hình 2.1: Biểu đồ use case tổng quát của hệ thống

Hình 2.1 thể hiện biểu đồ use case tổng quát của hệ thống với hai tác nhân chính: Người dùng và Quản trị viên. Mỗi tác nhân tương ứng với một nhóm chức năng riêng biệt.

Nhóm chức năng Người dùng gồm bốn use case mức cao: (i) *Xem dashboard chất lượng không khí*, hiển thị các thông số AQI, PM2.5, PM10, CO₂, TVOC, nhiệt độ, độ ẩm của trạm đo gần nhất theo thời gian thực; (ii) *Xem lịch sử dữ liệu*, tra cứu biểu đồ lịch sử theo trạm, theo thông số và theo khoảng thời gian; (iii) *Xem bản đồ AQI*, hiển thị bản đồ toàn khu vực với các trạm đo được đánh dấu bằng mã màu AQI; và (iv) *Xem xếp hạng ô nhiễm*, sắp xếp các trạm theo mức AQI từ cao đến thấp.

Nhóm chức năng Quản trị viên gồm năm use case mức cao: (i) *Quản lý thiết bị*, thêm, sửa, xóa và giám sát trạng thái Gateway và Sensor Node; (ii) *Giám sát cảnh báo*, xem danh sách cảnh báo, lọc theo mức độ nghiêm trọng, xác nhận đã xử lý; (iii) *Cấu hình hệ thống*, thiết lập ngưỡng cảnh báo cho từng thông số và chu kỳ gửi dữ liệu; (iv) *Quản lý tài khoản*, thêm, sửa, xóa tài khoản người dùng và phân quyền admin/user; và (v) *Xuất dữ liệu*, xuất dữ liệu đo lường ra tệp CSV theo trạm và khoảng thời gian.

2.2.2 Biểu đồ use case phân rã nhóm chức năng Người dùng



Hình 2.2: Biểu đồ use case phân rã nhóm chức năng Người dùng

Hình 2.2 thể hiện biểu đồ phân rã bốn use case của nhóm chức năng Người dùng. Mỗi use case được phân rã thành các chức năng con cụ thể như sau.

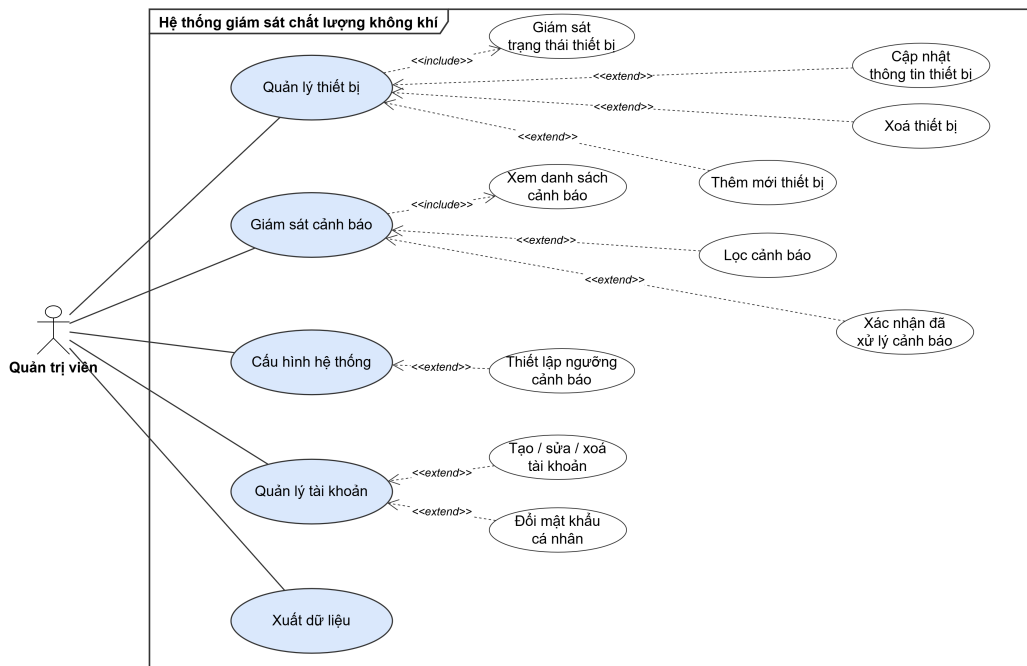
Xem dashboard chất lượng không khí. Khi người dùng truy cập dashboard, hệ thống tự động gợi ý trạm đo gần nhất dựa trên vị trí GPS thông qua Geolocation API và PostGIS. Dashboard hiển thị sáu thông số môi trường gồm PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm, cùng chỉ số AQI tổng hợp được tính theo công thức của US EPA [8]. Ngoài ra, hệ thống hiển thị cảnh báo sức khỏe theo hướng dẫn của WHO [2], kèm khuyến nghị cho nhóm nhạy cảm. Dữ liệu được cập nhật thời gian thực thông qua Socket.IO.

Xem lịch sử dữ liệu. Chức năng này cho phép người dùng: (i) chọn trạm đo cần xem, (ii) chọn thông số cần tra cứu (AQI, PM2.5, PM10, CO₂, TVOC, nhiệt độ hoặc độ ẩm), và (iii) chọn chế độ xem theo giờ hoặc theo ngày. Kết quả được hiển thị dưới dạng biểu đồ sử dụng thư viện ECharts, giúp người dùng quan sát xu hướng thay đổi chất lượng không khí theo thời gian.

Xem bản đồ AQI. Hệ thống hiển thị bản đồ toàn khu vực giám sát sử dụng thư viện Leaflet. Các trạm đo được đánh dấu bằng chấm tròn với mã màu AQI theo chuẩn US EPA [8]: xanh (tốt), vàng (trung bình), cam (không tốt cho nhóm nhạy cảm), đỏ (có hại), tím (rất có hại) và nâu đỏ (nguy hiểm). Người dùng có thể nhấn vào trạm để xem thông tin chi tiết.

Xem xếp hạng ô nhiễm. Chức năng hiển thị danh sách các trạm được sắp xếp theo mức AQI từ cao đến thấp. Mỗi dòng hiển thị tên trạm, mức AQI, nồng độ PM2.5 và màu sắc tương ứng, giúp người dùng nhanh chóng xác định khu vực ô nhiễm nhất.

2.2.3 Biểu đồ use case phân rã nhóm chức năng Quản trị



Hình 2.3: Biểu đồ use case phân rã nhóm chức năng Quản trị

Hình 2.3 thể hiện biểu đồ phân rã năm use case của nhóm chức năng Quản trị. Mỗi use case được phân rã thành các thao tác cụ thể như sau.

Quản lý thiết bị. Quản trị viên có thể thêm mới Gateway bằng cách nhập tên và mô tả vị trí, trong đó hệ thống tự sinh ID theo định dạng GW_XXX. Tương tự, quản trị viên thêm mới Sensor Node bằng cách nhập tên, chọn Gateway cha và nhập tọa độ GPS, hệ thống tự sinh ID theo định dạng NODE_XXX. Quản trị viên cũng có thể cập nhật thông tin thiết bị (tên, vị trí, trạng thái) và xóa thiết bị. Ngoài ra, hệ thống hỗ trợ giám sát trạng thái thiết bị, hiển thị trạng thái online/offline, thời gian gửi dữ liệu cuối (last seen) và mức pin.

Giám sát cảnh báo. Quản trị viên xem danh sách cảnh báo với hỗ trợ phân trang, có thể lọc theo loại (tên thiết bị), mức độ (cảnh báo hoặc nguy hiểm) và trạng thái xác nhận. Quản trị viên có thể xác nhận đã xử lý cảnh báo (acknowledge). Hệ thống phát thông báo cảnh báo thời gian thực qua Socket.IO khi có sự kiện mới.

Cấu hình hệ thống. Quản trị viên thiết lập ngưỡng cảnh báo cho sáu thông số: PM2.5, PM10, CO₂, TVOC, nhiệt độ (tối thiểu/tối đa). Mỗi thông số có hai mức: cảnh báo (warn) và nguy hiểm (danger).

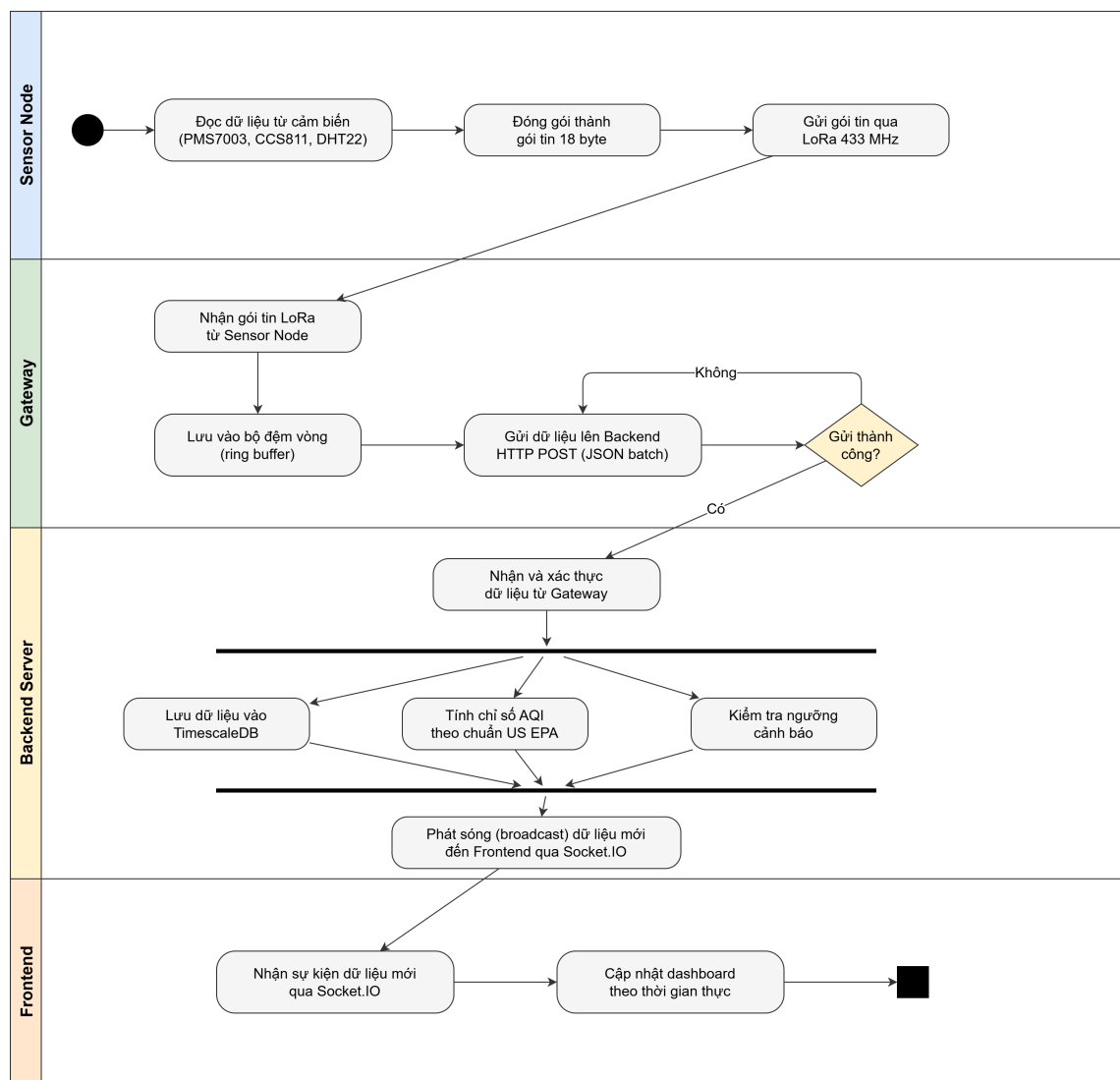
Quản lý tài khoản. Quản trị viên thực hiện tạo, sửa, xóa tài khoản người dùng và phân quyền admin hoặc user. Mỗi người dùng có thể đổi mật khẩu cá nhân. Hệ

thống bảo vệ không cho xóa tài khoản admin cuối cùng nhằm đảm bảo luôn có ít nhất một quản trị viên trong hệ thống.

2.2.4 Quy trình nghiệp vụ

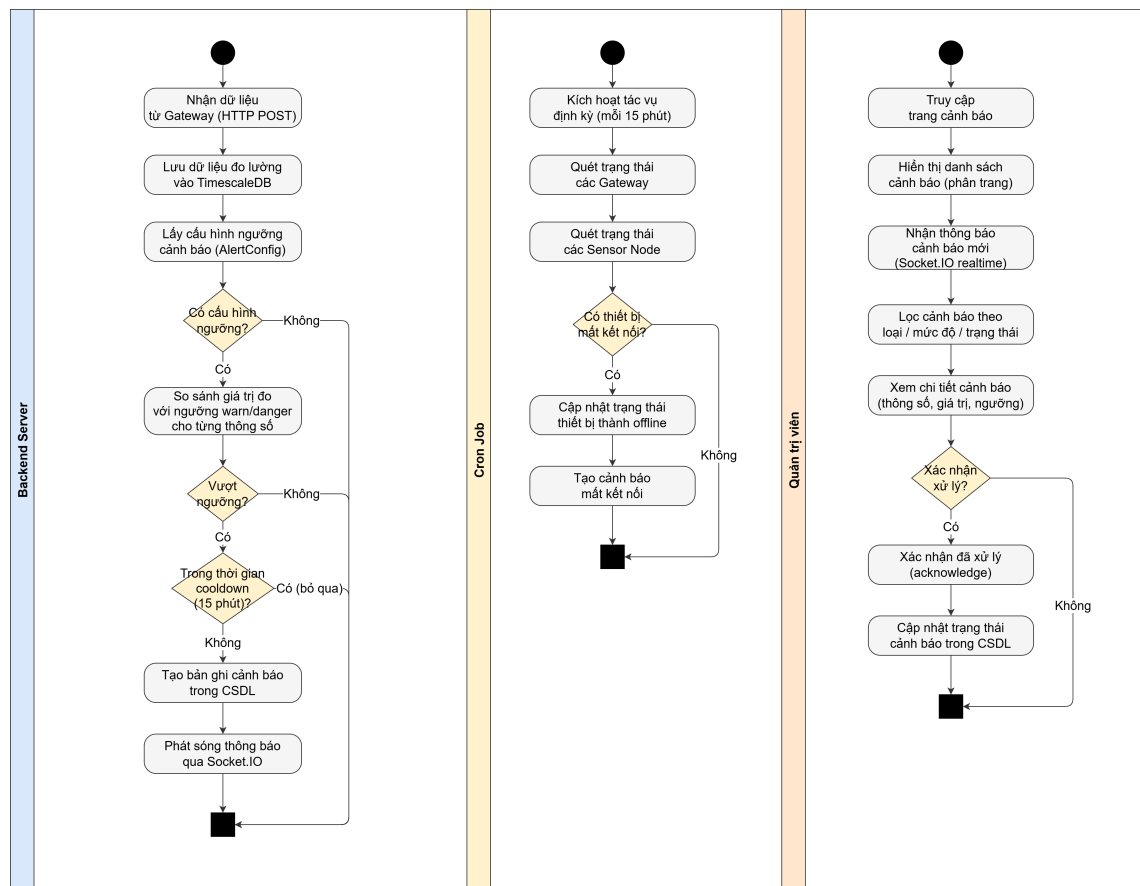
Hệ thống có hai quy trình nghiệp vụ chính.

Quy trình 1: Thu thập và hiển thị dữ liệu thời gian thực. Sensor Node đọc dữ liệu từ các cảm biến (PMS7003, CCS811, DHT22), đóng gói thành gói tin 18 byte và gửi qua LoRa đến Gateway. Gateway nhận gói tin, lưu vào bộ đệm vòng (ring buffer), sau đó gửi lên Backend Server dưới dạng HTTP POST (JSON batch). Backend Server xử lý dữ liệu: lưu vào cơ sở dữ liệu TimescaleDB, tính chỉ số AQI theo công thức US EPA [8], kiểm tra ngưỡng cảnh báo, và phát sóng (broadcast) dữ liệu mới đến tất cả Frontend thông qua Socket.IO. Frontend cập nhật dashboard theo thời gian thực mà không cần người dùng tải lại trang. Hình 2.4 minh họa toàn bộ luồng xử lý của quy trình này, từ khi Sensor Node đọc dữ liệu đến khi Frontend hiển thị kết quả.



Hình 2.4: Biểu đồ hoạt động quy trình thu thập và hiển thị dữ liệu

Quy trình 2: Phát hiện và xử lý cảnh báo. Khi Backend Server nhận dữ liệu mới từ Gateway, hệ thống tự động so sánh giá trị đo được với ngưỡng cảnh báo đã cấu hình. Nếu giá trị vượt ngưỡng, hệ thống tạo bản ghi cảnh báo trong cơ sở dữ liệu và phát sóng thông báo đến giao diện quản trị qua Socket.IO. Ngoài ra, một tác vụ định kỳ (cron job) chạy mỗi 5 phút kiểm tra các thiết bị không gửi dữ liệu trong thời gian quy định, tự động tạo cảnh báo mất kết nối. Quản trị viên xem danh sách cảnh báo, lọc theo tiêu chí, và xác nhận đã xử lý. Hình 2.5 minh họa luồng xử lý cảnh báo từ khi phát hiện bất thường đến khi quản trị viên xác nhận.



Hình 2.5: Biểu đồ hoạt động quy trình phát hiện và xử lý cảnh báo

2.3 Đặc tả chức năng

Phần này đặc tả chi tiết năm use case quan trọng nhất của hệ thống. Mỗi đặc tả bao gồm tên use case, tác nhân, mô tả, luồng sự kiện chính, luồng phát sinh, tiền điều kiện và hậu điều kiện.

2.3.1 Đặc tả use case Xem dashboard chất lượng không khí

Bảng 2.2: Đặc tả use case Xem dashboard chất lượng không khí

Tên use case	Xem dashboard chất lượng không khí
Tác nhân	Người dùng
Mô tả	Người dùng truy cập dashboard để xem các thông số chất lượng không khí theo thời gian thực tại trạm đo gần nhất.
Tiền điều kiện	Có ít nhất một trạm đo đang hoạt động trong hệ thống.
Luồng sự kiện chính	1. Người dùng truy cập trang dashboard. 2. Hệ thống yêu cầu quyền truy cập vị trí (Geolocation API). 3. Người dùng cho phép truy cập vị trí. 4. Hệ thống truy vấn trạm đo gần nhất dựa trên toạ độ GPS (PostGIS). 5. Hệ thống hiển thị sáu thông số môi trường (PM2.5, PM10, CO₂, TVOC, nhiệt độ, độ ẩm), chỉ số AQI tổng hợp, cảnh báo sức khoẻ WHO và thông tin thời tiết. 6. Hệ thống tự động cập nhật dữ liệu khi có dữ liệu mới từ Socket.IO.
Luồng phát sinh	3a. Người dùng từ chối truy cập vị trí: hệ thống hiển thị dữ liệu của trạm đo mặc định. 4a. Không có trạm nào đang hoạt động: hệ thống hiển thị thông báo “Không có dữ liệu”.
Hậu điều kiện	Người dùng xem được dữ liệu chất lượng không khí thời gian thực.

2.3.2 Đặc tả use case Xem lịch sử dữ liệu

Bảng 2.3: Đặc tả use case Xem lịch sử dữ liệu

Tên use case	Xem lịch sử dữ liệu
Tác nhân	Người dùng
Mô tả	Người dùng tra cứu dữ liệu lịch sử của một trạm đo theo thông số và khoảng thời gian mong muốn.
Tiền điều kiện	Trạm đo đã có dữ liệu đo lường trong cơ sở dữ liệu.
Luồng sự kiện chính	1. Người dùng chọn trạm đo từ danh sách. 2. Người dùng chọn thông số cần xem (AQI, PM2.5, PM10, CO₂, TVOC, nhiệt độ hoặc độ ẩm). 3. Người dùng chọn chế độ xem: theo giờ hoặc theo ngày. 4. Hệ thống truy vấn dữ liệu lịch sử từ TimescaleDB (Continuous Aggregate cho chế độ theo giờ). 5. Hệ thống hiển thị biểu đồ thể hiện xu hướng thay đổi theo thời gian.
Luồng phát sinh	4a. Không có dữ liệu trong khoảng thời gian được chọn: hệ thống hiển thị thông báo “Không có dữ liệu” và biểu đồ trống.
Hậu điều kiện	Người dùng xem được biểu đồ lịch sử dữ liệu.

2.3.3 Đặc tả use case Quản lý thiết bị

Bảng 2.4: Đặc tả use case Quản lý thiết bị

Tên use case	Quản lý thiết bị (Gateway và Sensor Node)
Tác nhân	Quản trị viên
Mô tả	Quản trị viên thực hiện thêm, sửa, xóa và giám sát trạng thái các thiết bị trong hệ thống.
Tiền điều kiện	Quản trị viên đã đăng nhập với quyền admin (JWT hợp lệ).
Luồng sự kiện chính	1. Quản trị viên truy cập trang quản lý thiết bị. 2. Hệ thống hiển thị bảng danh sách Gateway và Sensor Node với trạng thái, thời gian gửi cuối, mức pin. 3. Quản trị viên chọn thao tác: thêm mới, sửa hoặc xóa thiết bị. 4. Hệ thống hiển thị biểu mẫu tương ứng (modal). 5. Quản trị viên nhập thông tin và xác nhận. 6. Hệ thống xác thực dữ liệu đầu vào, lưu vào cơ sở dữ liệu, và cập nhật bảng danh sách.
Luồng phát sinh	6a. Dữ liệu đầu vào không hợp lệ: hệ thống hiển thị thông báo lỗi cụ thể.
Hậu điều kiện	Danh sách thiết bị được cập nhật trong cơ sở dữ liệu. Hệ thống ghi lại thao tác vào nhật ký kiểm toán (audit log).

2.3.4 Đặc tả use case Giám sát và xử lý cảnh báo

Bảng 2.5: Đặc tả use case Giám sát và xử lý cảnh báo

Tên use case	Giám sát và xử lý cảnh báo
Tác nhân	Quản trị viên
Mô tả	Quản trị viên xem, lọc và xác nhận các cảnh báo do hệ thống tự động tạo khi phát hiện bất thường.
Tiền điều kiện	Quản trị viên đã đăng nhập. Hệ thống đã cấu hình ngưỡng cảnh báo.
Luồng sự kiện chính	1. Quản trị viên truy cập trang cảnh báo. 2. Hệ thống hiển thị danh sách cảnh báo (phân trang, mới nhất trước). 3. Quản trị viên lọc cảnh báo theo loại (tên thiết bị), mức độ (cảnh báo/nguy hiểm), trạng thái xác nhận. 4. Quản trị viên chọn một cảnh báo để xác nhận đã xử lý. 5. Hệ thống cập nhật trạng thái cảnh báo thành “đã xác nhận”.
Luồng phát sinh	2a. Khi có cảnh báo mới (realtime): hệ thống tự động hiển thị thông báo trên giao diện qua Socket.IO mà không cần tải lại trang.
Hậu điều kiện	Cảnh báo được cập nhật trạng thái trong cơ sở dữ liệu.

2.3.5 Đặc tả use case Xuất dữ liệu CSV

Bảng 2.6: Đặc tả use case Xuất dữ liệu CSV

Tên use case	Xuất dữ liệu CSV
Tác nhân	Quản trị viên
Mô tả	Quản trị viên xuất dữ liệu đo lường ra tệp CSV để phân tích ngoại tuyến hoặc lưu trữ.
Tiền điều kiện	Quản trị viên đã đăng nhập. Có dữ liệu đo lường trong cơ sở dữ liệu.
Luồng sự kiện chính	1. Quản trị viên truy cập trang xuất dữ liệu. 2. Quản trị viên chọn Sensor Node cần xuất dữ liệu. 3. Quản trị viên chọn khoảng thời gian (ngày bắt đầu, ngày kết thúc). 4. Quản trị viên nhấn nút “Xuất CSV”. 5. Hệ thống truy vấn dữ liệu từ cơ sở dữ liệu, chuyển đổi sang định dạng CSV. 6. Trình duyệt tải xuống tệp CSV.
Luồng phát sinh	5a. Không có dữ liệu trong khoảng thời gian được chọn: hệ thống hiển thị thông báo “Không có dữ liệu để xuất”.
Hậu điều kiện	Tệp CSV được tải xuống máy tính quản trị viên.

2.4 Yêu cầu phi chức năng

Ngoài các yêu cầu chức năng đã đặc tả ở trên, hệ thống cần đáp ứng các yêu cầu phi chức năng được trình bày trong Bảng 2.7.

Bảng 2.7: Các yêu cầu phi chức năng của hệ thống

STT	Yêu cầu	Mô tả
1	Hiệu năng	Dữ liệu thời gian thực được cập nhật trên dashboard trong vòng 2 giây kể từ khi Gateway gửi dữ liệu lên server. Truy vấn lịch sử 30 ngày dữ liệu phải phản hồi trong vòng 3 giây nhờ TimescaleDB Continuous Aggregate.
2	Độ tin cậy	Hệ thống hoạt động liên tục 24/7. Cơ chế phát hiện thiết bị mất kết nối tự động (cron job định kỳ 5 phút). Gateway có bộ đệm vòng và cơ chế gửi lại (retry) khi mất kết nối server.
3	Khả năng mở rộng	Kiến trúc hub-spoke cho phép mở rộng số lượng Sensor Node mà không thay đổi phần mềm. Mỗi Gateway hỗ trợ nhiều Sensor Node. Hệ thống hỗ trợ thêm Gateway mới thông qua cơ chế tự đăng ký (provisioning).
4	Tính dễ sử dụng	Dashboard web có giao diện trực quan, responsive trên nhiều kích thước màn hình. Hệ thống hỗ trợ đa ngôn ngữ (Tiếng Việt và Tiếng Anh).
5	Bảo mật	Xác thực quản trị viên bằng JWT. Mật khẩu được mã hoá bằng bcrypt. Mọi thao tác quản trị được ghi nhận trong nhật ký kiểm toán (audit log) gồm người thực hiện, thời gian, nội dung thay đổi và địa chỉ IP.

Chương này đã khảo sát bốn hệ thống giám sát chất lượng không khí tiêu biểu, phân tích ưu nhược điểm và xác định rõ khoảng trống mà các hệ thống hiện tại chưa giải quyết được. Trên cơ sở đó, hệ thống đề xuất kết hợp cảm biến chi phí thấp, truyền thông LoRa tầm xa, mã nguồn mở và dashboard web thời gian thực. Phân tích yêu cầu đã xác định đầy đủ các chức năng cho hai tác nhân (Người dùng và Quản trị viên), đặc tả chi tiết năm use case quan trọng nhất, và định nghĩa sáu yêu cầu phi chức năng. Các yêu cầu này là cơ sở để lựa chọn nền tảng công nghệ phù hợp, được trình bày trong Chương 3.

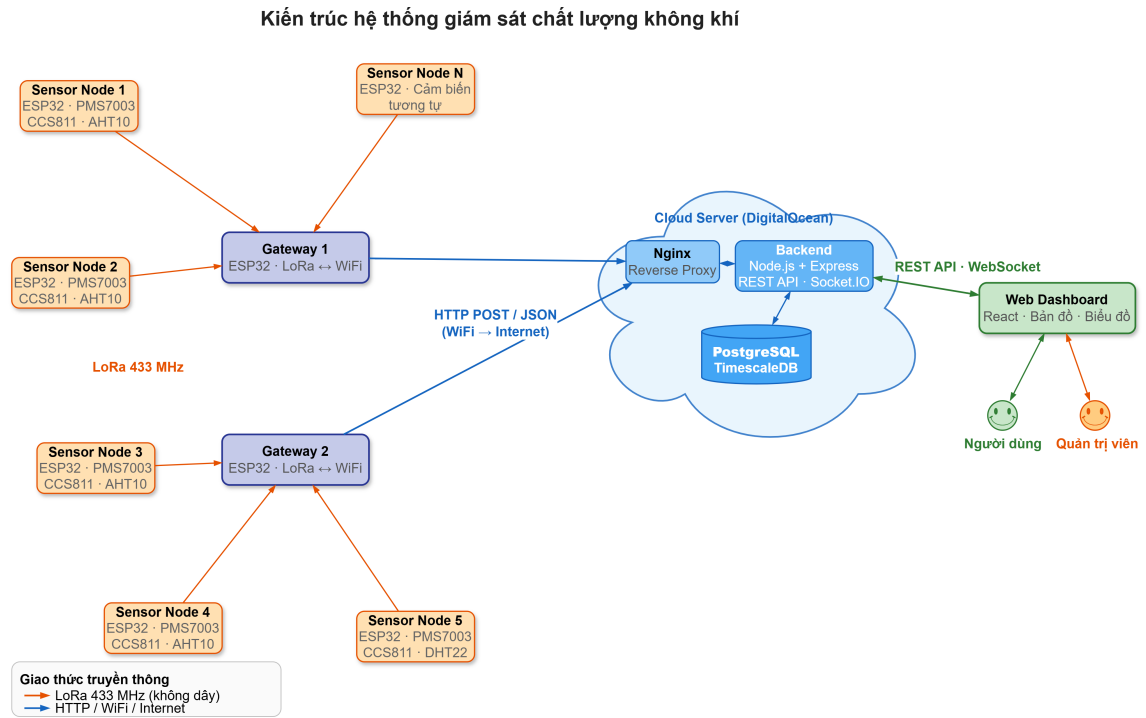
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định đầy đủ các yêu cầu chức năng và phi chức năng của hệ thống giám sát chất lượng không khí, bao gồm thu thập dữ liệu đa thông số từ cảm biến chi phí thấp, truyền dữ liệu tầm xa qua LoRa, xử lý dữ liệu chuỗi thời gian, hiển thị dashboard thời gian thực và quản trị hệ thống. Trên cơ sở đó, Chương 3 trình bày nền tảng lý thuyết và các công nghệ được lựa chọn để hiện thực hóa hệ thống. Nội dung chương bao gồm: (i) kiến trúc tổng thể trong phần 3.1, (ii) nền tảng phần cứng Internet vạn vật (Internet of Things – IoT) trong phần 3.2, (iii) giao thức truyền thông Long Range (LoRa) trong phần 3.3, (iv) công nghệ phần mềm phía máy chủ trong phần 3.4, (v) công nghệ phần mềm phía giao diện trong phần 3.5, (vi) lý thuyết tính toán chỉ số chất lượng không khí trong phần 3.6, và (vii) hạ tầng triển khai trong phần 3.7.

3.1 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo kiến trúc bốn tầng, bao gồm tầng cảm biến, tầng trung chuyển, tầng máy chủ và tầng hiển thị. Tầng cảm biến gồm các Sensor Node, mỗi node tích hợp vi điều khiển ESP32, các module cảm biến môi trường và module LoRa. Tầng trung chuyển là Gateway, đóng vai trò cầu nối giữa mạng LoRa và mạng Internet thông qua kết nối WiFi. Tầng máy chủ bao gồm Backend Server xử lý nghiệp vụ và cơ sở dữ liệu. Tầng hiển thị là ứng dụng web cung cấp giao diện cho người dùng và quản trị viên.

Luồng dữ liệu của hệ thống diễn ra như sau. Sensor Node đọc dữ liệu từ các cảm biến, đóng gói thành gói tin nhị phân 18 byte và phát qua LoRa. Gateway nhận gói tin, lưu vào bộ đệm vòng và gửi lên Backend Server dưới dạng HTTP POST (JSON batch). Backend Server lưu dữ liệu vào cơ sở dữ liệu TimescaleDB, tính chỉ số Chỉ số chất lượng không khí (Air Quality Index – AQI), kiểm tra ngưỡng cảnh báo và phát sóng đến giao diện web thông qua Socket.IO. Kiến trúc này đảm bảo tính module hóa: mỗi tầng có thể phát triển và mở rộng độc lập mà không ảnh hưởng đến các tầng còn lại.



Hình 3.1: Kiến trúc tổng thể hệ thống giám sát chất lượng không khí

3.2 Nền tảng phần cứng IoT

3.2.1 Vi điều khiển ESP32

ESP32 là dòng vi điều khiển do Espressif Systems phát triển, tích hợp bộ xử lý hai nhân Xtensa LX6 32-bit hoạt động ở tần số tối đa 240 MHz, bộ nhớ SRAM 520 KB, WiFi 802.11 b/g/n và Bluetooth 4.2 [9]. Vi điều khiển này hỗ trợ đầy đủ các giao tiếp ngoại vi cần thiết cho hệ thống: UART để giao tiếp với module LoRa và cảm biến PMS7003, và I2C để giao tiếp với cảm biến CCS811, AHT10 và màn hình OLED.

Đồ án sử dụng board ESP32 DevKit V1 với framework Arduino trên nền tảng PlatformIO. Firmware của Sensor Node sử dụng FreeRTOS — Hệ điều hành thời gian thực (Real-Time Operating System – RTOS) tích hợp sẵn trong ESP32 — để quản lý đồng thời các tác vụ đọc cảm biến, gửi dữ liệu LoRa và hiển thị OLED. Firmware của Gateway sử dụng kiến trúc superloop do chỉ cần thực hiện hai tác vụ tuần tự: nhận gói tin LoRa và chuyển tiếp lên Backend Server qua WiFi.

So với các lựa chọn thay thế, Arduino Mega không tích hợp WiFi nên không thể thực hiện provisioning qua web. STM32 có hiệu năng tương đương nhưng thiếu WiFi tích hợp và hệ sinh thái thư viện cảm biến ít phong phú hơn. Raspberry Pi có năng lực xử lý vượt trội nhưng chi phí cao gấp ba đến năm lần và tiêu thụ năng lượng lớn, không phù hợp cho node cảm biến hoạt động bằng pin.

3.2.2 Module cảm biến

Hệ thống sử dụng ba loại cảm biến để đo sáu thông số môi trường theo yêu cầu đặt ra ở Chương 2. Tất cả các cảm biến đều giao tiếp qua giao tiếp nối tiếp (UART hoặc I2C), giúp đơn giản hóa thiết kế phần cứng.

PMS7003 (Plantower) là cảm biến laser đo nồng độ bụi mịn PM2.5 và PM10 trong không khí [10]. Cảm biến hoạt động theo nguyên lý tán xạ laser: luồng không khí được hút qua buồng đo, các hạt bụi đi qua chùm laser sẽ tạo ra tín hiệu tán xạ, từ đó vi xử lý tích hợp tính toán nồng độ theo đơn vị $\mu g/m^3$. PMS7003 giao tiếp qua UART với tốc độ 9600 baud, kích thước nhỏ gọn ($48 \times 37 \times 12$ mm) và chi phí khoảng 350.000 VNĐ. Lựa chọn thay thế bao gồm PMS5003 cùng hãng Plantower với thông số tương tự nhưng tỉ lệ hỏng cao hơn, và SDS011 (Nova Fitness) có kích thước lớn hơn đáng kể ($71 \times 70 \times 23$ mm).

CCS811 là cảm biến chất lượng không khí sử dụng công nghệ oxit kim loại (MOX), đo đồng thời nồng độ CO₂ tương đương (eCO₂, phạm vi 400–8192 ppm) và tổng hợp chất hữu cơ bay hơi (Tổng các hợp chất hữu cơ bay hơi (Total Volatile Organic Compounds – TVOC), phạm vi 0–1187 ppb) [11]. Cảm biến giao tiếp qua I2C, chi phí khoảng 150.000 VNĐ. Lựa chọn thay thế bao gồm SGP30 (Sensirion) có thông số tương đương và SCD30 (Sensirion) sử dụng công nghệ NDIR cho độ chính xác cao hơn nhưng chi phí đắt gấp năm lần, vượt mục tiêu chi phí thấp của đề tài.

AHT10 (Aosong Electronics) là cảm biến đo nhiệt độ (phạm vi -40 đến 85°C , sai số $\pm 0.3^\circ\text{C}$) và độ ẩm tương đối (phạm vi 0–100% RH, sai số $\pm 2\%$ RH) [12]. Cảm biến sử dụng giao tiếp I2C (địa chỉ 0x38), chia sẻ chung bus I2C với CCS811, giúp giảm số chân GPIO cần sử dụng trên ESP32. Chi phí rất thấp (khoảng 35.000 VNĐ). Lựa chọn thay thế gồm DHT22 (cùng hãng Aosong, giao tiếp 1-wire, sai số $\pm 0.5^\circ\text{C}$) và SHT31 (Sensirion) có độ chính xác cao hơn ($\pm 0.2^\circ\text{C}$) nhưng chi phí gấp năm lần.

Bảng 3.1 tổng hợp thông số kỹ thuật của các cảm biến được sử dụng trong hệ thống.

Bảng 3.1: Thông số kỹ thuật các cảm biến sử dụng trong hệ thống

Cảm biến	Thông số đo	Phạm vi	Giao tiếp	Chi phí
PMS7003	PM2.5, PM10	0–500 $\mu\text{g}/\text{m}^3$	UART	~350.000 VNĐ
CCS811	eCO ₂ , eTVOC	400–8192 ppm, 0–1187 ppb	I2C	~150.000 VNĐ
AHT10	Nhiệt độ, độ ẩm	–40–85°C, 0–100% RH	I2C	~35.000 VNĐ

3.3 Giao thức truyền thông LoRa

3.3.1 Tổng quan về LoRa

LoRa (Long Range) là kỹ thuật điều chế lớp vật lý do Semtech phát triển, sử dụng phương pháp trải phổ Chirp Spread Spectrum (CSS) trên băng tần sub-GHz không cần cấp phép (433 MHz, 868 MHz hoặc 915 MHz tùy khu vực) [6]. Đặc điểm nổi bật của LoRa là khả năng truyền tầm xa (5–15 km trong điều kiện tầm nhìn thẳng) với mức tiêu thụ năng lượng rất thấp, tuy nhiên tốc độ truyền dữ liệu bị giới hạn (0,3–37,5 kbps). Đặc tính này phù hợp với bài toán giám sát môi trường, nơi khối lượng dữ liệu nhỏ (18 byte mỗi gói tin) nhưng yêu cầu phạm vi phủ sóng rộng.

Cần phân biệt LoRa (lớp vật lý) với LoRaWAN (giao thức mạng). LoRaWAN định nghĩa kiến trúc mạng và giao thức truyền thông trên nền LoRa, bao gồm quản lý thiết bị, mã hóa và kiến trúc hình sao. Đồ án sử dụng LoRa ở chế độ point-to-point (không qua LoRaWAN) để đơn giản hóa kiến trúc và giảm chi phí, vì hệ thống chỉ cần truyền dữ liệu từ Sensor Node đến Gateway trong phạm vi một khu vực (campus hoặc khu đô thị).

Kết quả khảo sát ở Chương 2 cho thấy tất cả bốn hệ thống hiện tại (PAM Air, IQAir, PurpleAir, VN-AQI) đều sử dụng WiFi hoặc 4G, yêu cầu hạ tầng mạng tại mỗi điểm đo. LoRa giải quyết hạn chế này bằng cách cho phép truyền dữ liệu tầm xa mà không cần WiFi tại mỗi node. Các lựa chọn thay thế bao gồm: (i) WiFi có tầm phủ ngắn (dưới 100 m) và cần điểm truy cập tại mỗi vị trí, (ii) Zigbee có tầm phủ ngắn (dưới 100 m) dù tiêu thụ năng lượng thấp, (iii) NB-IoT và 4G yêu cầu SIM và phí dữ liệu hàng tháng, và (iv) Sigfox phụ thuộc vào nhà mạng Sigfox chưa phủ sóng tại Việt Nam.

3.3.2 Module AS32-TTL-100

Đồ án sử dụng module AS32-TTL-100 dựa trên chip SX1278 (Semtech), hoạt động ở tần số 433 MHz ISM band với công suất phát 100 mW (20 dBm). Module giao tiếp với vi điều khiển qua UART ở chế độ transparent — nghĩa là dữ liệu ghi

vào cổng UART sẽ được module tự động điều chế và phát qua sóng LoRa, và ngược lại. Chế độ này đơn giản hóa đáng kể firmware so với việc sử dụng chip SX1278 trực tiếp qua SPI, vốn yêu cầu cấu hình thành ghi phức tạp.

Module hỗ trợ bốn chế độ hoạt động được điều khiển qua hai chân MD0 và MD1: chế độ truyền bình thường (MD0=0, MD1=0), chế độ tiết kiệm năng lượng, chế độ cấu hình và chế độ ngủ (MD0=1, MD1=1). Khoảng cách truyền lý thuyết đạt 3–5 km trong môi trường đô thị và trên 10 km trong điều kiện tầm nhìn thẳng.

3.4 Công nghệ phần mềm phía máy chủ

3.4.1 Node.js và Express

Node.js là môi trường thực thi JavaScript phía máy chủ, xây dựng trên engine V8 của Google Chrome [13]. Đặc điểm quan trọng nhất của Node.js là mô hình event-driven, non-blocking I/O: thay vì tạo luồng mới cho mỗi kết nối (như mô hình truyền thống), Node.js sử dụng vòng lặp sự kiện đơn luồng để xử lý đồng thời hàng nghìn kết nối. Mô hình này phù hợp với hệ thống giám sát thời gian thực, nơi Backend cần duy trì nhiều kết nối Socket.IO đồng thời và xử lý dữ liệu telemetry liên tục từ các Gateway.

Express là framework web phổ biến nhất cho Node.js, cung cấp hệ thống middleware, routing và xử lý HTTP request/response. Đồ án sử dụng Express để xây dựng REST API cho các chức năng CRUD thiết bị, truy vấn dữ liệu, xuất CSV và quản lý tài khoản. Các middleware bảo mật bao gồm Helmet (thiết lập HTTP security headers), CORS (kiểm soát truy cập cross-origin) và xác thực JWT.

So với các lựa chọn thay thế: Python kết hợp Flask hoặc FastAPI có hiệu năng hạn chế hơn cho kịch bản real-time do mô hình đồng thời khác biệt. Java kết hợp Spring Boot có kiến trúc vững chắc nhưng chi phí tài nguyên và độ phức tạp cao cho dự án quy mô nhỏ. Go kết hợp Gin có hiệu năng vượt trội nhưng hệ sinh thái thư viện nhỏ hơn đáng kể.

3.4.2 PostgreSQL, TimescaleDB và PostGIS

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, tuân thủ ACID và hỗ trợ hệ thống extension phong phú [14]. Đồ án mở rộng PostgreSQL bằng hai extension chuyên biệt: TimescaleDB và PostGIS.

TimescaleDB là extension chuyển đổi PostgreSQL thành cơ sở dữ liệu chuỗi thời gian (time-series), được thiết kế cho dữ liệu có nhãn thời gian liên tục [15]. Đồ án sử dụng ba tính năng chính của TimescaleDB. Thứ nhất, *Hypertable*: TimescaleDB tự động phân mảnh (partition) bảng dữ liệu theo thời gian, giúp tối ưu hiệu năng ghi và truy vấn khi lượng dữ liệu tăng liên tục. Thứ hai, *Continuous Aggregate*:

materialized view được tự động cập nhật khi có dữ liệu mới, cho phép truy vấn lịch sử tổng hợp với hiệu năng cao mà không cần tính toán lại từ dữ liệu thô — đáp ứng yêu cầu phi chức năng truy vấn 30 ngày trong vòng 3 giây (Bảng 2.7). Thứ ba, *Retention Policy*: tự động xóa dữ liệu cũ theo chu kỳ cấu hình để kiểm soát dung lượng lưu trữ.

PostGIS là extension bổ sung kiểu dữ liệu không gian và các hàm xử lý địa lý cho PostgreSQL [16]. PostGIS cho phép lưu trữ tọa độ GPS dưới dạng đối tượng hình học và thực hiện các phép tính khoảng cách địa lý, từ đó hỗ trợ truy vấn trạm đo gần nhất dựa trên vị trí người dùng — đáp ứng yêu cầu chức năng tự động gợi ý trạm gần nhất trên dashboard (Bảng 2.2). Chi tiết cách áp dụng TimescaleDB và PostGIS vào thiết kế cơ sở dữ liệu sẽ được trình bày trong Chương 4.

So với các lựa chọn thay thế: InfluxDB chuyên biệt cho time-series nhưng không hỗ trợ dữ liệu không gian, buộc phải sử dụng hai cơ sở dữ liệu riêng biệt. MongoDB là cơ sở dữ liệu NoSQL linh hoạt nhưng không tối ưu cho truy vấn time-series aggregate. MySQL thiếu extension tương đương TimescaleDB và PostGIS.

3.4.3 Prisma ORM

Prisma là ORM thế hệ mới cho Node.js, áp dụng phương pháp schema-first: lược đồ cơ sở dữ liệu được định nghĩa trong một file khai báo, từ đó Prisma Client tự động sinh ra các hàm truy vấn type-safe [17].

Ưu điểm chính của Prisma so với viết SQL thủ công là giảm lỗi runtime nhờ kiểm tra kiểu tại thời điểm biên dịch, hỗ trợ migration tự động khi thay đổi lược đồ, và cú pháp truy vấn trực quan. Ngoài ra, Prisma cho phép kết hợp với raw SQL khi cần sử dụng các tính năng đặc thù của extension (TimescaleDB, PostGIS) mà ORM chưa hỗ trợ trực tiếp.

3.4.4 Socket.IO

Socket.IO là thư viện giao tiếp thời gian thực hai chiều giữa máy chủ và trình duyệt, xây dựng trên nền WebSocket với cơ chế fallback về HTTP long-polling khi WebSocket không khả dụng [18]. Đề án sử dụng Socket.IO cho hai chức năng: (i) phát sóng (broadcast) dữ liệu đo mới đến tất cả giao diện dashboard khi Backend nhận dữ liệu từ Gateway, và (ii) phát thông báo cảnh báo thời gian thực đến giao diện quản trị khi phát hiện giá trị vượt ngưỡng.

So với WebSocket thuần, Socket.IO cung cấp thêm cơ chế tự động kết nối lại (auto-reconnect), hệ thống phòng (room) để phát sóng có chọn lọc và xử lý graceful degradation. So với Server-Sent Events (SSE), Socket.IO hỗ trợ giao tiếp hai chiều, cần thiết cho các tính năng tương tác trong tương lai.

3.5 Công nghệ phần mềm phía giao diện

3.5.1 React và Vite

React là thư viện xây dựng giao diện người dùng do Meta phát triển, dựa trên kiến trúc component — mỗi thành phần giao diện là một đơn vị độc lập, có thể tái sử dụng và kết hợp để tạo nên giao diện phức tạp [19]. React sử dụng Virtual DOM để tối ưu hiệu năng cập nhật giao diện: khi trạng thái thay đổi, React so sánh Virtual DOM mới với phiên bản trước, chỉ cập nhật các phần tử thực sự thay đổi trên DOM thật. Cơ chế này phù hợp với dashboard thời gian thực, nơi dữ liệu liên tục được cập nhật qua Socket.IO nhưng chỉ một số thành phần giao diện cần vẽ lại.

Vite là công cụ build thế hệ mới, sử dụng ES Module trên trình duyệt trong quá trình phát triển và Rollup để đóng gói sản phẩm [20]. Vite cung cấp Hot Module Replacement (HMR) gần như tức thì, tăng tốc đáng kể quy trình phát triển so với Webpack truyền thống.

So với các lựa chọn thay thế: Vue.js nhẹ hơn nhưng hệ sinh thái thư viện bên thứ ba nhỏ hơn. Angular là framework toàn diện nhưng có đường cong học tập dốc và chi phí tài nguyên cao cho dự án quy mô nhỏ. Svelte có hiệu năng runtime tốt nhưng hệ sinh thái chưa trưởng thành.

3.5.2 Thư viện trực quan hóa dữ liệu

Đồ án sử dụng hai thư viện trực quan hóa chính. **Leaflet** là thư viện bản đồ tương tác mã nguồn mở, nhẹ (khoảng 42 KB gzip) và hỗ trợ đầy đủ các tính năng bản đồ: marker, popup, zoom và lớp phủ [21]. Đồ án sử dụng Leaflet kết hợp React-Leaflet để hiển thị bản đồ AQI với marker mã màu theo chuẩn Cơ quan Bảo vệ Môi trường Hoa Kỳ (United States Environmental Protection Agency – US EPA) tại mỗi trạm đo. So với Google Maps API (phải trả phí sau số lượng request nhất định) và Mapbox (freemium, yêu cầu API key), Leaflet hoàn toàn miễn phí và sử dụng tile từ OpenStreetMap.

Apache ECharts là thư viện biểu đồ hiệu năng cao, hỗ trợ hơn 20 loại biểu đồ và khả năng xử lý tập dữ liệu lớn [22]. Đồ án sử dụng ECharts thông qua wrapper `echarts-for-react` để hiển thị biểu đồ lịch sử dữ liệu theo giờ và theo ngày. So với Chart.js (nhẹ hơn nhưng ít tính năng nâng cao), ECharts hỗ trợ tốt hơn cho responsive tương tác phức tạp.

3.6 Lý thuyết tính toán chỉ số chất lượng không khí

3.6.1 Khái niệm và phân loại mức AQI

Chỉ số chất lượng không khí (AQI) là thước đo tổng hợp mức độ ô nhiễm không khí, được US EPA định nghĩa trên thang điểm từ 0 đến 500 [8]. Giá trị AQI càng

cao thì mức ô nhiễm càng nghiêm trọng và ảnh hưởng sức khỏe càng lớn. US EPA phân chia thang AQI thành sáu mức như trình bày trong Bảng 3.2.

Bảng 3.2: Phân loại các mức chỉ số AQI theo tiêu chuẩn US EPA [8]

AQI	Mức độ	Mã màu	Ảnh hưởng sức khỏe
0–50	Tốt (Good)	Xanh lá	Không ảnh hưởng
51–100	Trung bình (Moderate)	Vàng	Nhóm nhạy cảm nên hạn chế
101–150	Không tốt cho nhóm nhạy cảm	Cam	Nhóm nhạy cảm bị ảnh hưởng
151–200	Có hại (Unhealthy)	Đỏ	Mọi người bắt đầu bị ảnh hưởng
201–300	Rất có hại (Very Unhealthy)	Tím	Cảnh báo sức khỏe nghiêm trọng
301–500	Nguy hiểm (Hazardous)	Nâu đỏ	Toàn bộ dân số bị ảnh hưởng

3.6.2 Công thức và bảng điểm ngắt

Đồ án tính AQI cho từng chất ô nhiễm dạng hạt (PM_{2.5}, PM₁₀) theo công thức nội suy tuyến tính của US EPA:

$$I_p = \frac{I_{Hi} - I_{Lo}}{BP_{Hi} - BP_{Lo}} \times (C_p - BP_{Lo}) + I_{Lo} \quad (3.1)$$

Trong đó I_p là giá trị AQI của chất ô nhiễm p , C_p là nồng độ đo được (đơn vị $\mu g/m^3$), BP_{Hi} và BP_{Lo} là các điểm ngắt nồng độ (breakpoint) trên và dưới chứa C_p , I_{Hi} và I_{Lo} là giá trị AQI tương ứng với các điểm ngắt đó. Chỉ số AQI tổng hợp được xác định bằng giá trị lớn nhất trong các AQI thành phần: $AQI = \max(I_{PM2.5}, I_{PM10})$.

Bảng 3.3 và Bảng 3.4 trình bày các điểm ngắt nồng độ cho PM_{2.5} và PM₁₀ theo tiêu chuẩn US EPA, được hệ thống sử dụng trực tiếp trong module tính toán trên Backend Server.

Bảng 3.3: Bảng điểm ngắt AQI cho PM_{2.5} [8]

$BP_{Lo} (\mu g/m^3)$	$BP_{Hi} (\mu g/m^3)$	I_{Lo}	I_{Hi}
0,0	12,0	0	50
12,1	35,4	51	100
35,5	55,4	101	150
55,5	150,4	151	200
150,5	250,4	201	300
250,5	500,4	301	500

Bảng 3.4: Bảng điểm ngắt AQI cho PM10 [8]

$BP_{Lo} (\mu g/m^3)$	$BP_{Hi} (\mu g/m^3)$	I_{Lo}	I_{Hi}
0	54	0	50
55	154	51	100
155	254	101	150
255	354	151	200
355	424	201	300
425	604	301	500

3.6.3 Ví dụ tính toán

Giả sử cảm biến đo được $C_{PM2.5} = 45,3 \mu g/m^3$ và $C_{PM10} = 82 \mu g/m^3$. Quy trình tính AQI diễn ra như sau.

Đối với PM2.5, tra Bảng 3.3: giá trị 45,3 nằm trong khoảng [35,5; 55,4], tương ứng $I_{Lo} = 101$, $I_{Hi} = 150$, $BP_{Lo} = 35,5$, $BP_{Hi} = 55,4$. Thay vào công thức (3.1):

$$I_{PM2.5} = \frac{150 - 101}{55,4 - 35,5} \times (45,3 - 35,5) + 101 = \frac{49}{19,9} \times 9,8 + 101 \approx 125$$

Đối với PM10, tra Bảng 3.4: giá trị 82 nằm trong khoảng [55; 154], tương ứng $I_{Lo} = 51$, $I_{Hi} = 100$, $BP_{Lo} = 55$, $BP_{Hi} = 154$. Thay vào công thức (3.1):

$$I_{PM10} = \frac{100 - 51}{154 - 55} \times (82 - 55) + 51 = \frac{49}{99} \times 27 + 51 \approx 64$$

Chỉ số AQI tổng hợp là $AQI = \max(125, 64) = 125$, thuộc mức “Không tốt cho nhóm nhạy cảm” (Bảng 3.2). Kết quả này cho thấy PM2.5 là chất ô nhiễm chính quyết định mức AQI, phù hợp với thực tế tại các đô thị Việt Nam, nơi bụi mịn PM2.5 thường là tác nhân ô nhiễm chủ đạo.

Hệ thống áp dụng quy trình tính toán này trên Backend Server mỗi khi nhận dữ liệu mới từ Gateway, đồng thời ánh xạ kết quả sang mã màu tương ứng (Bảng 3.2) để hiển thị trên bản đồ và dashboard.

3.7 Hạ tầng triển khai

3.7.1 Docker và Docker Compose

Docker là nền tảng container hóa cho phép đóng gói ứng dụng cùng toàn bộ phụ thuộc (thư viện, runtime, cấu hình) vào một đơn vị triển khai nhất quán gọi là container [23]. Docker Compose mở rộng Docker cho ứng dụng đa container, cho phép định nghĩa và quản lý toàn bộ hệ thống trong một file `docker-compose.yml`.

Đồ án container hóa bốn thành phần: (i) cơ sở dữ liệu PostgreSQL/TimescaleDB, (ii) Backend Server (Node.js), (iii) Nginx phục vụ static files và reverse proxy, và (iv) Certbot quản lý chứng chỉ SSL. Việc container hóa đảm bảo môi trường nhất quán giữa phát triển và sản xuất, đơn giản hóa quy trình triển khai trên VPS chỉ với một lệnh duy nhất. So với triển khai trực tiếp trên VPS (cài đặt phụ thuộc thủ công, khó tái tạo), Docker loại bỏ hoàn toàn vấn đề “hoạt động trên máy em nhưng không hoạt động trên server”. So với Kubernetes, Docker Compose đơn giản và phù hợp hơn cho hệ thống quy mô nhỏ chạy trên một máy chủ duy nhất.

3.7.2 Nginx

Nginx đóng vai trò reverse proxy, chuyển tiếp các yêu cầu HTTP/WebSocket từ client đến Backend Server, đồng thời phục vụ các tệp tĩnh (static files) của Frontend đã được build bởi Vite. Kiến trúc này giúp Backend Server không bị lộ trực tiếp ra Internet, tăng cường bảo mật và cho phép cấu hình HTTPS tập trung tại Nginx.

Chương này đã trình bày nền tảng lý thuyết và các công nghệ được lựa chọn cho từng tầng trong kiến trúc hệ thống. Ở tầng cảm biến, vi điều khiển ESP32 kết hợp ba module cảm biến (PMS7003, CCS811, AHT10) cho phép đo sáu thông số môi trường với chi phí thấp. Tầng trung chuyển sử dụng giao thức LoRa qua module AS32-TTL-100 giải quyết bài toán truyền dữ liệu tầm xa mà không cần hạ tầng WiFi tại mỗi điểm đo. Ở tầng máy chủ, Node.js kết hợp TimescaleDB và Socket.IO đáp ứng yêu cầu xử lý dữ liệu chuỗi thời gian và cập nhật thời gian thực. Ở tầng hiển thị, React kết hợp Leaflet và ECharts cung cấp dashboard trực quan. Toàn bộ hệ thống được container hóa bằng Docker để đơn giản hóa triển khai. Với các công nghệ đã lựa chọn, Chương 4 sẽ trình bày chi tiết thiết kế kiến trúc, triển khai các chức năng và đánh giá kết quả thực nghiệm.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương 3 đã trình bày nền tảng lý thuyết và các công nghệ được lựa chọn cho từng tầng trong kiến trúc hệ thống. Trên cơ sở đó, Chương 4 trình bày chi tiết quá trình thiết kế, triển khai và đánh giá hệ thống. Nội dung chương bao gồm thiết kế kiến trúc (phần 4.1), thiết kế chi tiết (phần 4.2), xây dựng ứng dụng (phần 4.3), kiểm thử (phần 4.4), và triển khai (phần 4.5).

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được thiết kế theo kiến trúc bốn tầng, bao gồm tầng cảm biến (Sensor Layer), tầng trung chuyển (Gateway Layer), tầng máy chủ (Server Layer) và tầng hiển thị (Presentation Layer). Kiến trúc bốn tầng được lựa chọn vì ba lý do chính. Thứ nhất, mỗi tầng có trách nhiệm rõ ràng và có thể phát triển độc lập: tầng cảm biến thu thập dữ liệu môi trường, tầng trung chuyển chuyển tiếp dữ liệu từ mạng LoRa sang Internet, tầng máy chủ xử lý nghiệp vụ và lưu trữ, tầng hiển thị cung cấp giao diện cho người dùng. Thứ hai, kiến trúc này cho phép mở rộng theo chiều ngang tại từng tầng — ví dụ thêm Sensor Node mới mà không thay đổi phần mềm Backend. Thứ ba, độ phức tạp phù hợp với quy mô hệ thống: triển khai trên một máy chủ VPS duy nhất, không yêu cầu kiến trúc Microservice.

Tầng cảm biến gồm các Sensor Node, mỗi node tích hợp vi điều khiển ESP32, ba module cảm biến (PMS7003, CCS811, AHT10) và module LoRa AS32-TTL-100. Firmware sử dụng FreeRTOS để quản lý đồng thời bốn tác vụ: đọc cảm biến, gửi dữ liệu LoRa, hiển thị OLED và đo pin. Dữ liệu được đóng gói thành gói tin nhị phân 18 byte và phát qua LoRa 433 MHz.

Tầng trung chuyển gồm các Gateway, mỗi Gateway tích hợp ESP32 với module LoRa và kết nối WiFi. Gateway nhận gói tin LoRa, lưu vào bộ đệm vòng (ring buffer), sau đó gửi lên tầng máy chủ dưới dạng HTTP POST (JSON batch). Ở tầng này, hệ thống áp dụng mô hình Hub-Spoke: nhiều Sensor Node (spoke) gửi dữ liệu đến một Gateway (hub) trung tâm, cho phép một Gateway phục vụ nhiều Sensor Node trong phạm vi phủ sóng mà không cần định tuyến phức tạp.

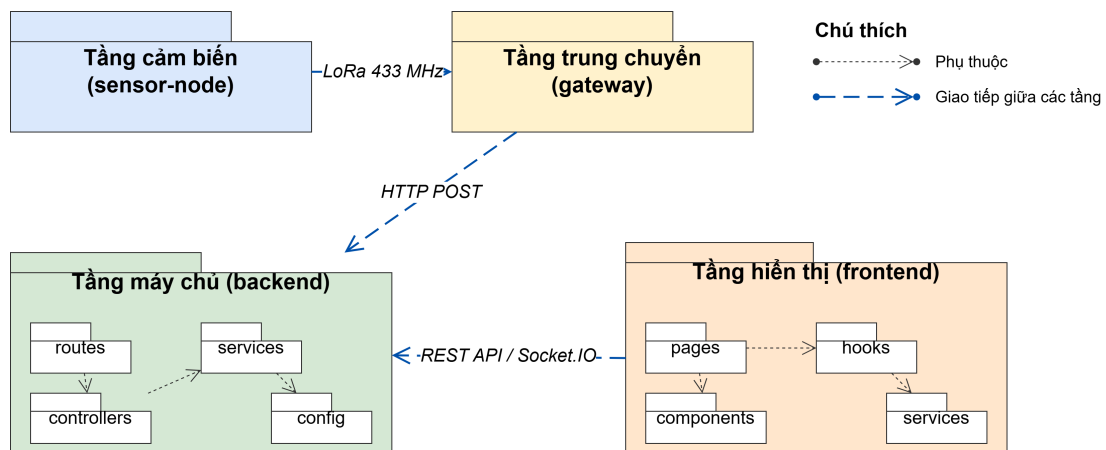
Tầng máy chủ bao gồm Backend Server (Node.js/Express) và cơ sở dữ liệu (PostgreSQL với TimescaleDB và PostGIS), được đóng gói trong Docker container. Backend Server được tổ chức theo mô hình phân lớp gồm ba lớp: lớp Controller tiếp nhận HTTP request, lớp Service chứa logic nghiệp vụ, và lớp Data Access

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

tương tác với cơ sở dữ liệu thông qua Prisma ORM. Ngoài ra, tầng máy chủ còn có Nginx đóng vai trò reverse proxy và phục vụ tệp tĩnh, cùng các tác vụ nền (cron job) kiểm tra thiết bị offline và dọn dẹp dữ liệu cũ.

Tầng hiển thị là ứng dụng web SPA xây dựng bằng React 19 và Vite 8, giao tiếp với tầng máy chủ qua REST API (HTTP/HTTPS) và Socket.IO (WebSocket). Tầng này cung cấp hai nhóm giao diện: dashboard cho người dùng (bản đồ AQI, biểu đồ lịch sử, xếp hạng) và trang quản trị cho quản trị viên (quản lý thiết bị, cảnh báo, cấu hình, xuất dữ liệu).

4.1.2 Thiết kế tổng quan



Hình 4.1: Biểu đồ gói UML tổng quan của hệ thống

Hệ thống được tổ chức thành bốn nhóm gói chính tương ứng với bốn tầng kiến trúc, mỗi nhóm chứa các gói con với trách nhiệm cụ thể.

Nhóm gói Tầng cảm biến (sensor-node) chứa các module: `drivers` (giao tiếp cảm biến PMS7003, CCS811, AHT10, màn hình OLED, module LoRa), `core` (quản lý cấu hình NVS, provisioning qua WiFi AP, vòng lặp chính), và `config` (định nghĩa chân GPIO, tham số LoRa). Hệ thống có ba biến thể firmware cho các loại board ESP32 khác nhau.

Nhóm gói Tầng trung chuyển (gateway) chứa các module: `drivers` (giao tiếp LoRa), `core` (bộ đệm vòng, gửi HTTP batch lên tầng máy chủ), và `config` (cấu hình WiFi, địa chỉ Backend). Nhóm gói này hoạt động độc lập, không phụ thuộc vào nhóm gói tầng cảm biến.

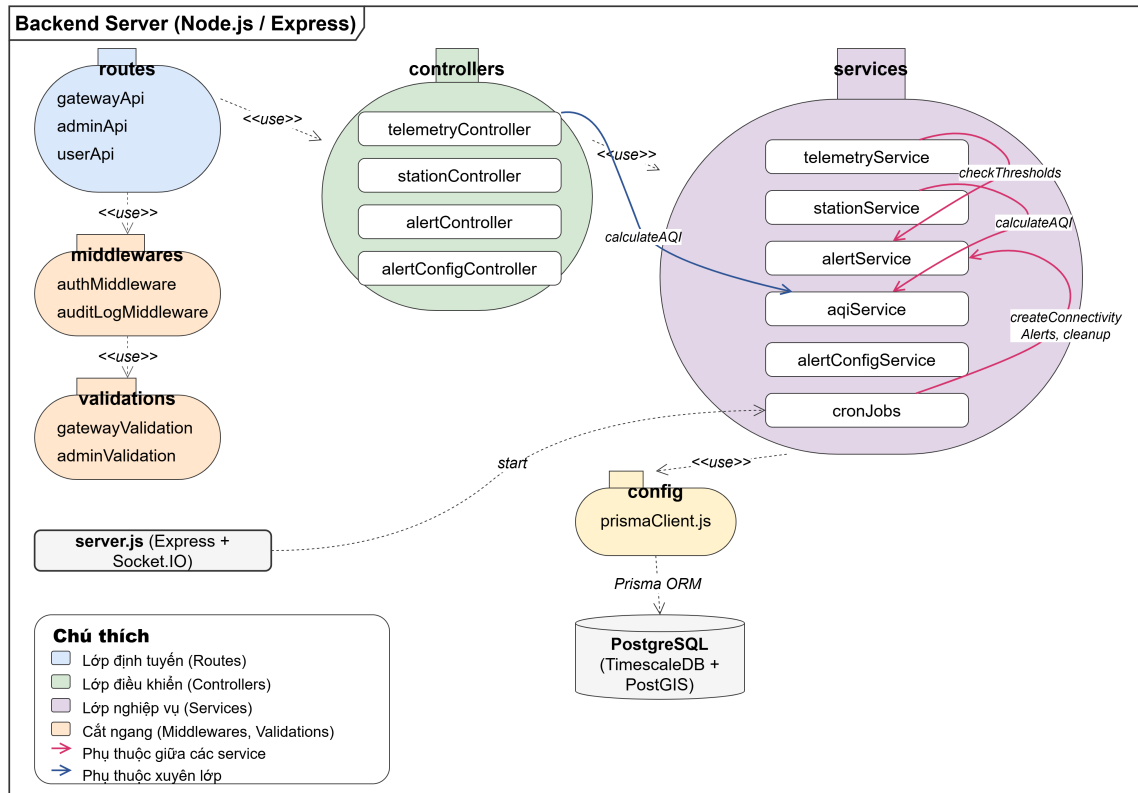
Nhóm gói Tầng máy chủ (backend) gồm sáu gói: `routes` (định nghĩa end-point API), `controllers` (tiếp nhận request, trả response), `services` (logic nghiệp vụ), `middlewares` (xác thực JWT, xử lý lỗi), `validations` (kiểm tra dữ liệu đầu vào), và `config` (kết nối Prisma Client). Luồng phụ thuộc tuân thủ nguyên tắc một chiều: `routes` → `controllers` → `services` → `config` (Prisma), đảm

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

bảo gói tầng dưới không phụ thuộc gói tầng trên.

Nhóm gói Tầng hiển thị (frontend) gồm bảy gói: pages (các trang giao diện), components (thành phần tái sử dụng), hooks (custom React hooks), services (gọi API), stores (quản lý trạng thái Zustand), layouts (bố cục trang), và i18n (đa ngôn ngữ). Gói pages phụ thuộc components và hooks; gói hooks phụ thuộc services; gói services gọi REST API đến tầng máy chủ.

4.1.3 Mô tả luồng dữ liệu giữa các gói



Hình 4.2: Biểu đồ quan hệ module cho backend services

Hình 4.2 mô tả quan hệ phụ thuộc giữa các module trên Backend. Hai nhóm module quan trọng nhất là nhóm xử lý dữ liệu telemetry và nhóm hiển thị dashboard.

Nhóm xử lý dữ liệu telemetry bao gồm bốn module: telemetryController tiếp nhận request, telemetryValidation xác thực đầu vào, telemetryService xử lý nghiệp vụ (deduplication, lưu trữ, kiểm tra cảnh báo), và aqiService tính chỉ số AQI. Luồng xử lý chi tiết được trình bày qua ba biểu đồ trình tự ở phần 4.2.4.

Nhóm hiển thị dashboard trên Frontend sử dụng hook useDashboard (gọi API lấy dữ liệu ban đầu) và hook useSocket (nhận cập nhật thời gian thực qua Socket.IO). Chi tiết luồng end-to-end từ truy cập Dashboard đến cập nhật realtime

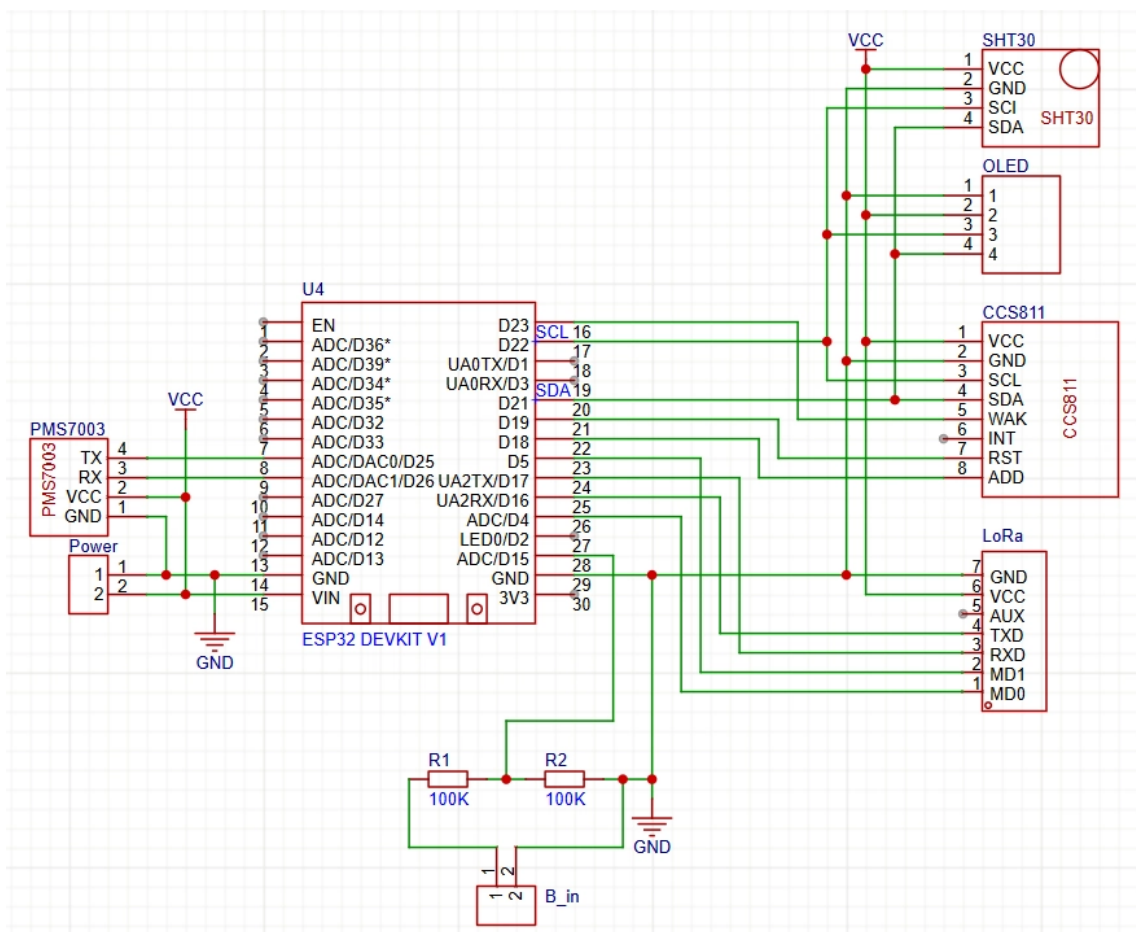
được mô tả ở Hình 4.6.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế phần cứng

Phần này trình bày thiết kế mạch kết nối phần cứng cho Sensor Node và Gateway.

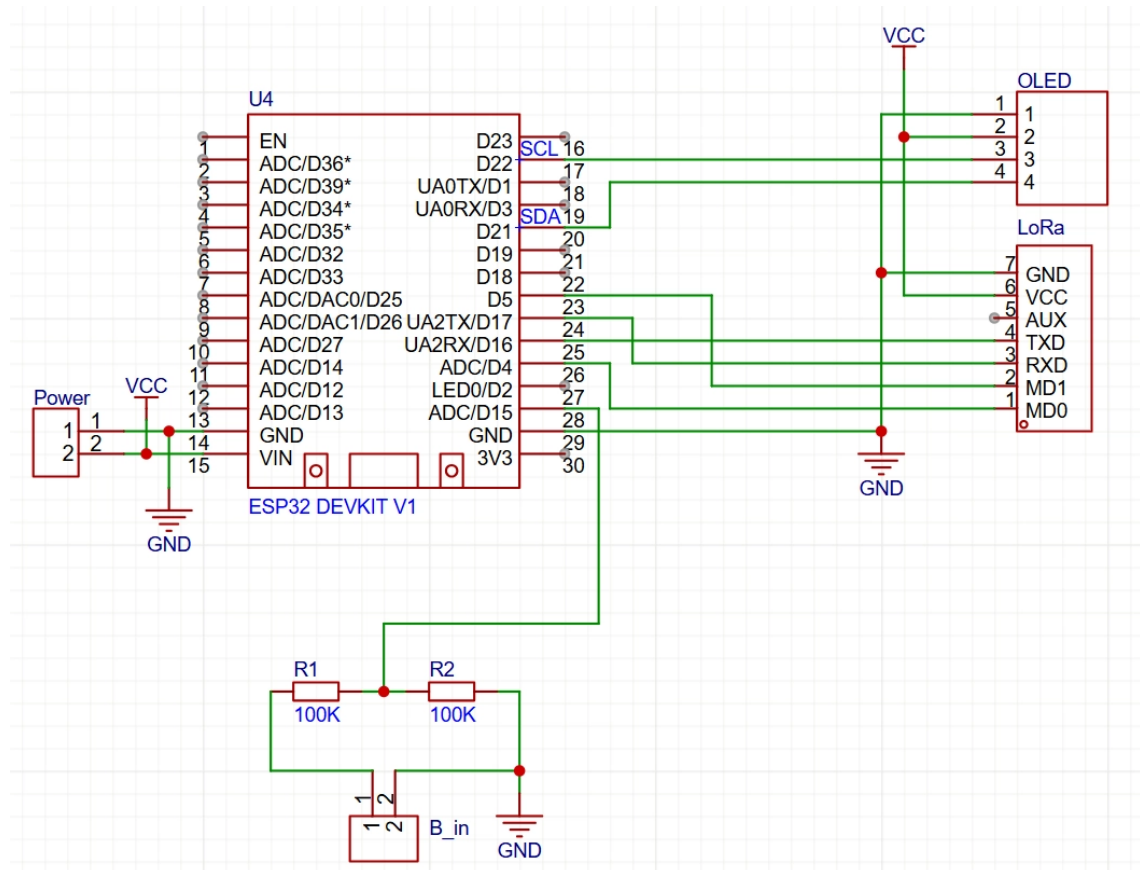
Sensor Node sử dụng board ESP32 DevKit V1 (30 chân) làm trung tâm, kết nối với bốn module ngoại vi qua ba giao tiếp khác nhau. Module LoRa AS32-TTL-100 giao tiếp qua UART1 (RX: GPIO16, TX: GPIO17) với hai chân điều khiển chế độ (MD0: GPIO4, MD1: GPIO5). Module cảm biến bụi PMS7003 giao tiếp qua UART2 (RX: GPIO25, TX: GPIO26). Module CCS811 và AHT10 chia sẻ chung bus I2C (SDA: GPIO21, SCL: GPIO22) với địa chỉ lần lượt là 0x5A và 0x38; CCS811 có thêm chân WAK (GPIO23) để đánh thức từ chế độ ngủ. Màn hình OLED SSD1306 cũng dùng chung bus I2C trên, địa chỉ 0x3C. Pin 18650 được đo qua mạch phân áp (voltage divider R1=R2=100K Ω) nối vào GPIO15 (ADC). Nút BOOT (GPIO0) được tận dụng làm nút factory reset: giữ 5 giây để xóa cấu hình và vào chế độ provisioning.



Hình 4.3: Sơ đồ mạch Sensor Node

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Gateway sử dụng cùng board ESP32 DevKit V1, nhưng chỉ kết nối một module ngoại vi: module LoRa AS32-TTL-100 qua UART1 (cùng cấu hình chân với Sensor Node). Gateway không cần cảm biến hay màn hình OLED, do chức năng duy nhất là nhận gói LoRa và chuyển tiếp lên tầng máy chủ qua WiFi. Thiết kế tối giản này giảm chi phí linh kiện và giảm tiêu thụ năng lượng cho Gateway.



Hình 4.4: Sơ đồ mạch Gateway

4.2.2 Thiết kế firmware

Phần này trình bày thiết kế firmware cho Sensor Node và Gateway.

Firmware Sensor Node sử dụng FreeRTOS với bốn tác vụ chạy song song trên hai nhân của ESP32. Tác vụ `sensor_task` (ưu tiên 2, Core 0, stack 4096 byte) đọc dữ liệu từ ba cảm biến theo chu kỳ 15 giây, thực hiện đến 3 lần retry khi đọc lỗi, sau đó đẩy dữ liệu vào hàng đợi FreeRTOS (queue depth = 3). Tác vụ `lora_task` (ưu tiên 3 — cao nhất, Core 1) nhận dữ liệu từ hàng đợi, đóng gói thành struct `SensorPayload` và phát qua module LoRa. Tác vụ `battery_task` (ưu tiên 1, Core 0) đọc pin mỗi 30 giây qua ADC với 16 mẫu lấy trung bình. Tác vụ `watch-dog_task` (ưu tiên 0, Core 0) giám sát hoạt động: nếu hệ thống không gửi dữ liệu trong 60 giây, tự động reset ESP32.

Cấu trúc gói tin LoRa được định nghĩa dưới dạng struct C packed (không có

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

padding), tổng kích thước 18 byte. Gói tin gồm: `nodeId` (1 byte, ID node 1–255), `pktType` (1 byte, phân biệt gói dữ liệu/heartbeat/lỗi), `msgId` (1 byte, bộ đếm tuần hoàn 0–255 phục vụ deduplication), bảy trường dữ liệu cảm biến (PM1, PM2.5, PM10 mỗi trường 2 byte nhân 10, CO₂ và TVOC mỗi trường 2 byte, nhiệt độ 2 byte có dấu nhân 10, độ ẩm 2 byte nhân 10), và `battery` (1 byte, 0–100%). Các giá trị dạng số thực được nhân 10 và lưu dạng số nguyên để tránh sử dụng floating-point trong truyền thông, giảm kích thước gói tin. Giá trị sentinel (0xFFFF cho unsigned, 0x7FFF cho signed) đánh dấu cảm biến bị lỗi.

Cơ chế provisioning cho phép cấu hình Node ID qua giao diện web. Khi chưa có cấu hình (hoặc sau factory reset), Sensor Node khởi động WiFi AP với SSID “AirQuality-SN-Setup”, phục vụ trang web captive portal tại địa chỉ 192.168.4.1. Người dùng nhập Node ID, firmware lưu vào bộ nhớ NVS (Non-Volatile Storage) và khởi động lại ở chế độ hoạt động bình thường.

Firmware Gateway sử dụng kiến trúc superloop (không dùng FreeRTOS) do chỉ thực hiện hai tác vụ tuần tự: nhận gói tin LoRa từ UART và tích lũy vào bộ đệm vòng (ring buffer). Khi đủ số lượng gói hoặc hết chu kỳ thời gian, Gateway serialize bộ đệm thành JSON array và gửi lên tầng máy chủ qua HTTP POST. Gateway cũng hỗ trợ cơ chế self-provisioning: tự đăng ký với Backend Server khi kết nối lần đầu.

4.2.3 Thiết kế giao diện

Giao diện hệ thống được thiết kế cho trình duyệt web, hỗ trợ responsive trên nhiều kích thước màn hình từ máy tính (độ phân giải tối thiểu 1280×720) đến thiết bị di động (tối thiểu 375×667). Hệ thống hỗ trợ hai ngôn ngữ (Tiếng Việt và Tiếng Anh) thông qua thư viện `react-i18next`.

Giao diện được tổ chức thành hai nhóm layout riêng biệt. **UserLayout** dành cho người dùng, gồm thanh điều hướng ngang (navbar) ở trên cùng với các liên kết đến bốn trang: Dashboard, Lịch sử, Bản đồ và Xếp hạng. **AdminLayout** dành cho quản trị viên, gồm thanh điều hướng dọc (sidebar) bên trái có thể thu gọn, chứa các mục: Dashboard, Gateways, Sensor Nodes, Cảnh báo, Cấu hình, Người dùng, Xuất dữ liệu và Nhật ký.

Hệ thống sử dụng bộ thành phần giao diện tái sử dụng (component library) tự xây dựng, bao gồm 28 component: Button, Modal, Badge, Input, Select, Table, Pagination, Card, Tooltip và các component chuyên biệt như AQIGauge, Metric-Card, HealthAdvisory. Tất cả component tuân thủ thiết kế nhất quán: dark theme với bảng màu chính là xanh dương đậm, các trạng thái tương tác (hover, focus, disabled) được định nghĩa thống nhất, và hiệu ứng chuyển đổi mượt mà (transition 200ms).

4.2.4 Thiết kế module Backend

Phần này trình bày thiết kế chi tiết cho bốn module quan trọng nhất trên Backend Server.

Module telemetryService chịu trách nhiệm xử lý dữ liệu đo lường từ Gateway. Module này cung cấp hai hàm chính: `processTelemetry(gateway_id, data)` xử lý gói dữ liệu batch và `processHeartbeat(gateway_id)` xử lý tín hiệu heartbeat. Hàm `processTelemetry` thực hiện tuần tự ba bước: lọc trùng lặp qua module `dedupCache`, thực thi transaction cập nhật trạng thái thiết bị và lưu dữ liệu đo lường, sau đó kiểm tra ngưỡng cảnh báo ngoài transaction.

Module aqiService triển khai thuật toán tính AQI theo chuẩn US EPA đã trình bày ở Chương 3. Module cung cấp năm hàm: `calculateSubAQI` tính AQI thành phần cho một chất ô nhiễm, `calculateAQI` tính AQI tổng hợp (giá trị lớn nhất giữa PM2.5 và PM10), `getAQIInfo` ánh xạ giá trị AQI sang mức và mã màu, `getCO2Info` và `getTVOCInfo` đánh giá mức CO₂ và TVOC. Bảng breakpoint được lưu trữ dưới dạng mảng hằng số trong mã nguồn, tương ứng chính xác với Bảng 3.3 và Bảng 3.4 ở Chương 3.

Module alertService quản lý toàn bộ vòng đời cảnh báo. Module cung cấp sáu hàm: `getAlerts` truy vấn danh sách cảnh báo với hỗ trợ lọc và phân trang, `acknowledgeAlert` xác nhận xử lý, `deleteAlert` xóa cảnh báo, `checkThresholdsAndCreateAlerts` kiểm tra năm thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ) so với ngưỡng cấu hình và tạo cảnh báo khi vượt ngưỡng, `createConnectivityAlerts` tạo cảnh báo mất kết nối cho thiết bị offline, và `cleanupOldAlerts` xóa cảnh báo quá 30 ngày. Cơ chế cooldown 15 phút ngăn chặn tạo cảnh báo trùng lặp cho cùng thiết bị và cùng thông số trong khoảng thời gian ngắn.

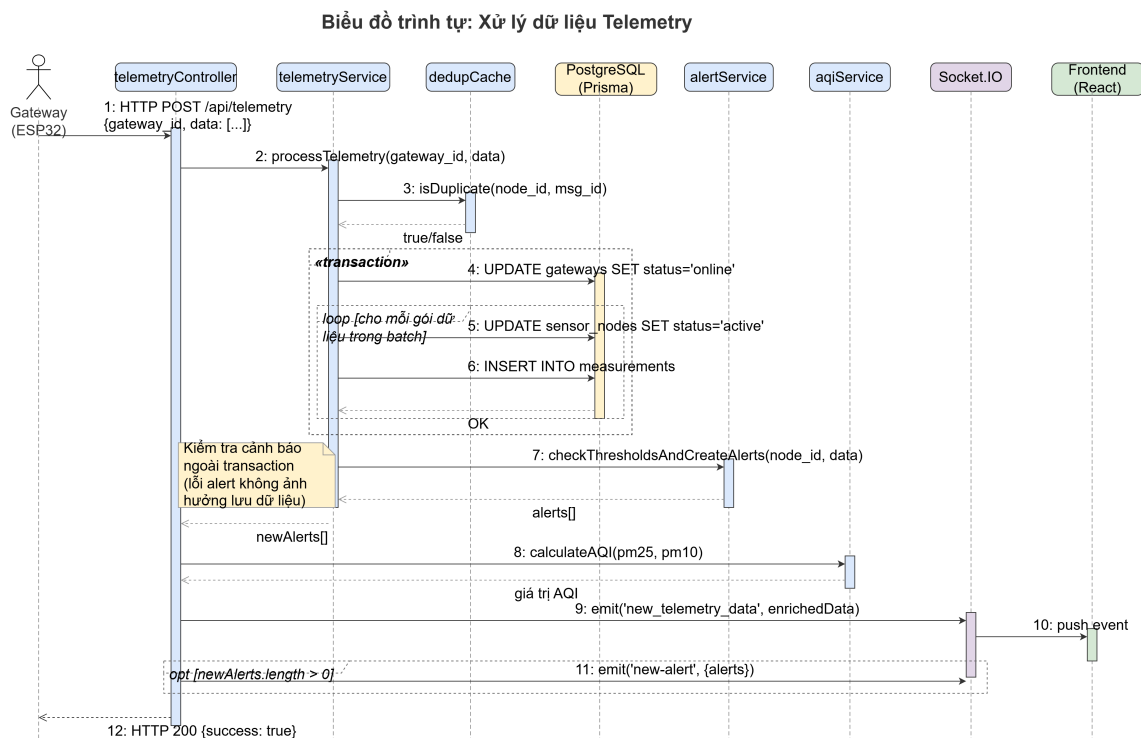
Module stationService cung cấp dữ liệu cho các trang người dùng. Hàm `getNearestStation` sử dụng PostGIS để truy vấn trạm đo gần nhất dựa trên toạ độ GPS của người dùng thông qua hàm `ST_Distance`. Hàm `getStationHistory` truy vấn dữ liệu lịch sử từ Continuous Aggregate `hourly_measurements` cho chế độ xem theo giờ, hoặc từ bảng `measurements` gốc cho chế độ xem theo ngày.

Để minh hoạ luồng tương tác giữa các thành phần, phần tiếp theo trình bày ba biểu đồ trình tự (sequence diagram) cho ba use case quan trọng nhất của hệ thống.

Biểu đồ trình tự 1: Xử lý dữ liệu Telemetry. Hình 4.5 mô tả luồng xử lý khi Gateway gửi dữ liệu đo lường lên Backend — đây là luồng cốt lõi của hệ thống

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

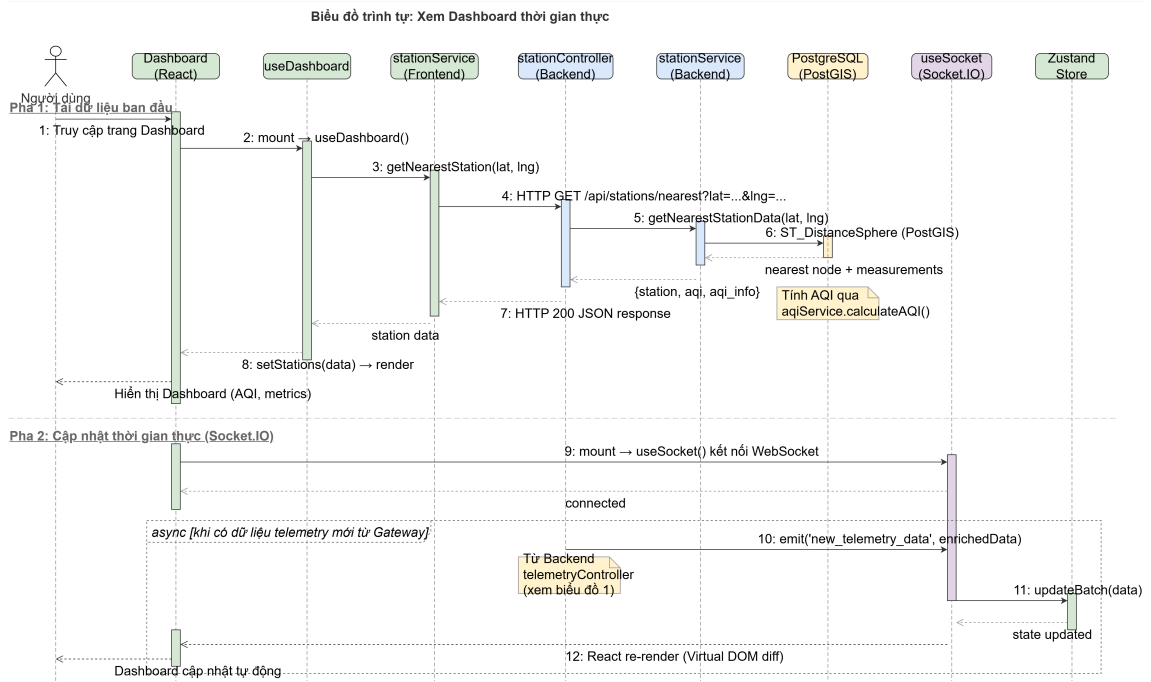
IoT. Gateway gửi HTTP POST chứa batch dữ liệu đến `telemetryController`. Controller chuyển tiếp cho `telemetryService`, module này thực hiện ba bước tuần tự: (i) lọc gói tin trùng lặp qua `dedupCache` (xử lý trường hợp nhiều Gateway cùng nhận một gói LoRa), (ii) thực thi database transaction để cập nhật trạng thái Gateway, cập nhật trạng thái Sensor Node và lưu dữ liệu đo lường vào bảng `measurements`, (iii) kiểm tra ngưỡng cảnh báo qua `alertService` — bước này chạy ngoài transaction để lỗi cảnh báo không ảnh hưởng đến luồng lưu dữ liệu chính. Sau đó, Controller tính chỉ số AQI qua `aqiService` và phát sự kiện `new_telemetry_data` qua Socket.IO đến Frontend. Nếu có cảnh báo mới, sự kiện `new-alert` cũng được phát để thông báo cho quản trị viên.



Hình 4.5: Biểu đồ trình tự xử lý dữ liệu Telemetry

Biểu đồ trình tự 2: Xem Dashboard thời gian thực. Hình 4.6 mô tả luồng end-to-end từ khi người dùng truy cập trang Dashboard đến khi nhận cập nhật thời gian thực. Luồng gồm hai pha. Pha 1 (tải dữ liệu ban đầu): khi React mount component Dashboard, hook `useDashboard` gọi API `/api/stations/nearest` kèm toạ độ GPS của người dùng. Backend sử dụng PostGIS (`ST_DistanceSphere`) để tìm trạm đo gần nhất, tính AQI và trả kết quả về Frontend để hiển thị. Pha 2 (cập nhật thời gian thực): hook `useSocket` kết nối WebSocket đến Backend. Mỗi khi có dữ liệu telemetry mới từ Gateway, Backend phát sự kiện `new_telemetry_data`; hook `useSocket` nhận sự kiện, cập nhật Zustand store, kích hoạt React Virtual DOM so sánh và chỉ vẽ lại các thành phần thay đổi trên giao diện.

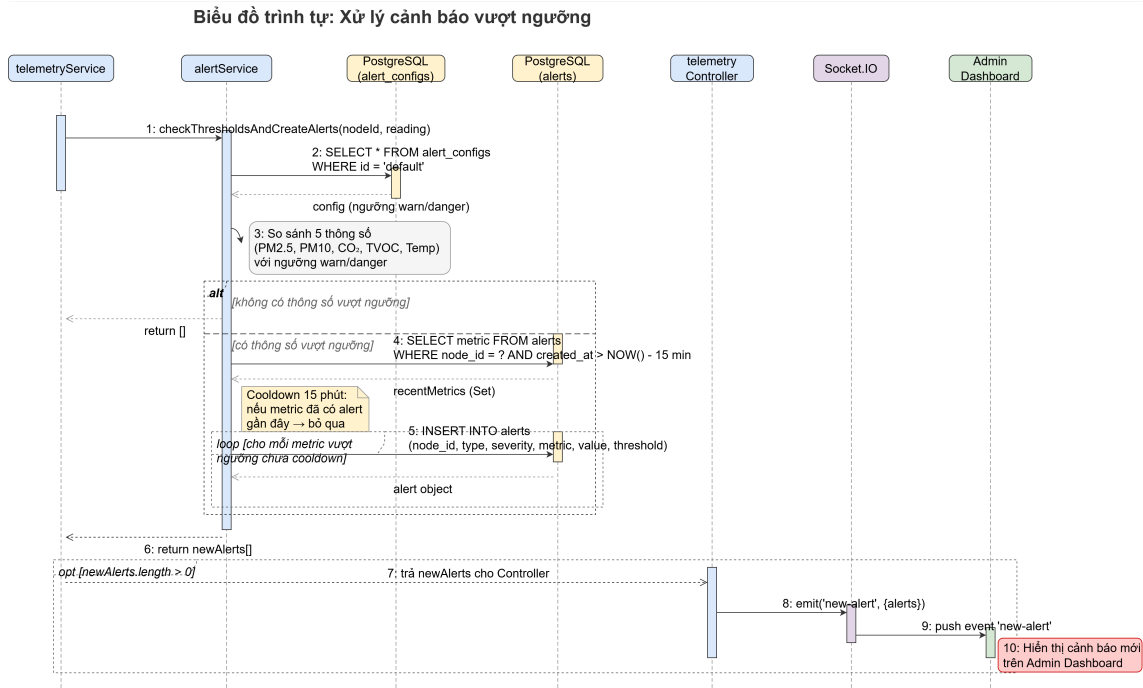
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.6: Biểu đồ trình tự xem Dashboard thời gian thực

Biểu đồ trình tự 3: Xử lý cảnh báo vượt ngưỡng. Hình 4.7 mô tả chi tiết luồng kiểm tra và tạo cảnh báo khi dữ liệu đo lường vượt ngưỡng cấu hình. Sau khi `telemetryService` lưu dữ liệu thành công, module gọi `alertService.checkThreshold`. Module `alertService` thực hiện bốn bước: (i) truy vấn bảng `alert_configs` để lấy ngưỡng warn/danger cho năm thông số (PM2.5, PM10, CO₂, TVOC, nhiệt độ), (ii) so sánh từng giá trị đo với ngưỡng tương ứng, (iii) kiểm tra cơ chế cooldown 15 phút — nếu cùng Sensor Node và cùng thông số đã có cảnh báo trong 15 phút gần đây thì bỏ qua để tránh tạo cảnh báo trùng lặp, (iv) tạo bản ghi cảnh báo mới trong bảng `alerts`. Kết quả trả về Controller, nếu có cảnh báo mới, Controller phát sự kiện `new-alert` qua Socket.IO để Admin Dashboard hiển thị thông báo tức thì.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.7: Biểu đồ trình tự xử lý cảnh báo vượt ngưỡng

4.2.5 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu được thiết kế trên PostgreSQL 15 với hai extension: TimescaleDB cho dữ liệu chuỗi thời gian và PostGIS cho dữ liệu không gian. Lược đồ gồm bảy bảng chính và một materialized view (trên biểu đồ Hình 4.8 chỉ thể hiện 7 bảng chính).

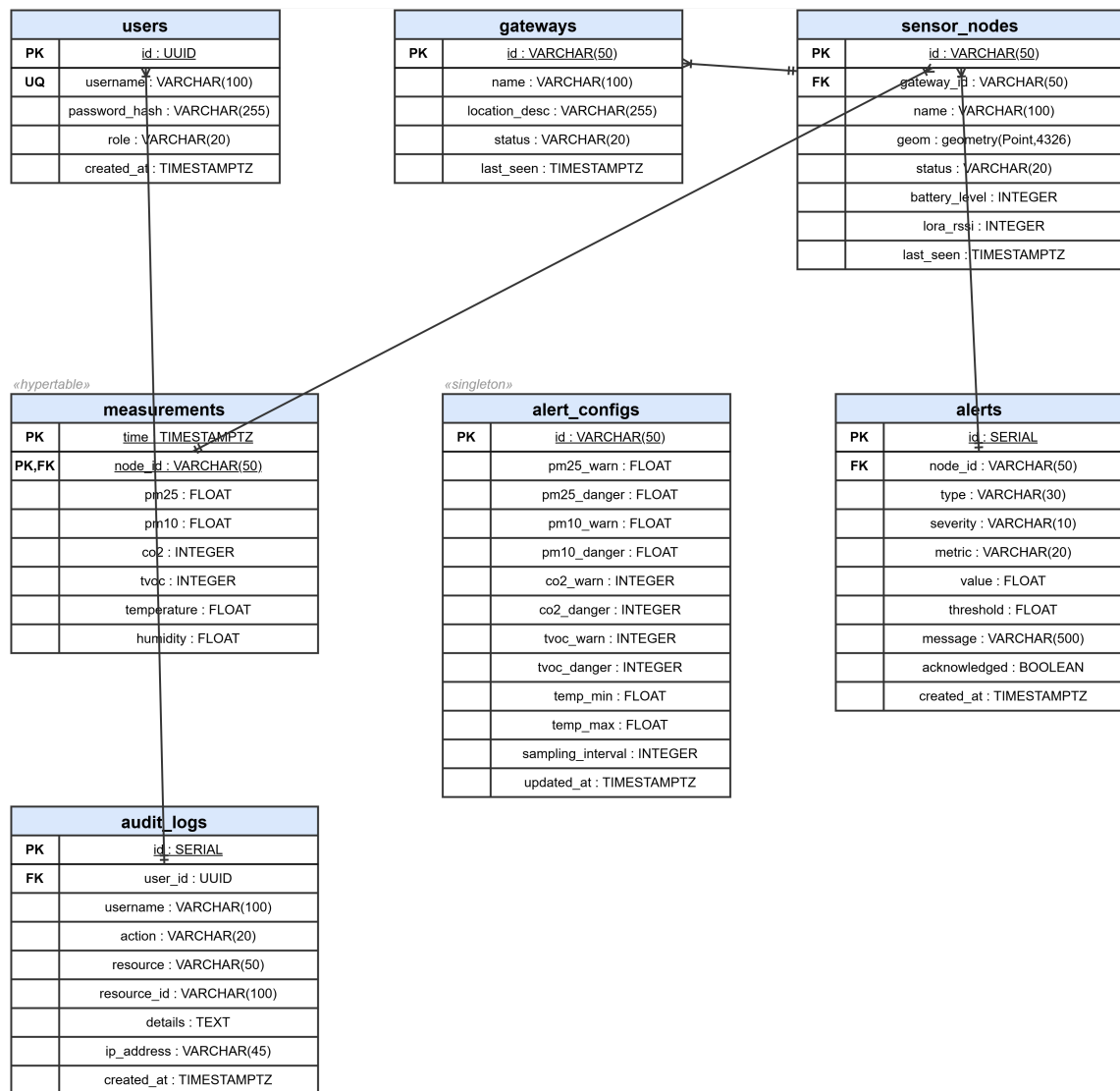
Bảng users lưu thông tin tài khoản với khóa chính là UUID tự sinh, bao gồm username (duy nhất), password_hash (mã hóa bcrypt) và role (admin hoặc user).

Bảng gateways lưu thông tin Gateway với khóa chính là mã định danh dạng chuỗi (ví dụ “GW_001”), bao gồm tên, mô tả vị trí, trạng thái online/offline và thời điểm gửi dữ liệu cuối (last_seen).

Bảng sensor_nodes lưu thông tin Sensor Node với khóa ngoại tham chiếu đến bảng gateways (quan hệ 1-N: một Gateway quản lý nhiều Sensor Node, xóa Gateway sẽ xóa cascade các Sensor Node). Cột geom kiểu geometry(Point, 4326) lưu tọa độ GPS theo hệ tọa độ WGS84, cho phép truy vấn không gian qua PostGIS. Các cột battery_level và lora_rssi lưu thông tin trạng thái phần cứng.

Bảng measurements là hypertable TimescaleDB, được tự động phân mảnh theo cột thời gian time. Khóa chính composite gồm (time, node_id) đảm bảo mỗi Sensor Node chỉ có một bản ghi tại mỗi thời điểm. Bảng lưu sáu thông số: PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm. Retention policy tự động xóa dữ liệu thô

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.8: Entity-Relationship Diagram

quá 3 tháng.

Materialized view hourly_measurements là Continuous Aggregate tự động tính giá trị trung bình theo giờ cho mỗi Sensor Node. View này được TimescaleDB tự động cập nhật khi có dữ liệu mới ghi vào bảng `measurements`, đáp ứng yêu cầu phi chức năng truy vấn lịch sử 30 ngày trong vòng 3 giây (Bảng 2.7).

Bảng alert_configs lưu cấu hình ngưỡng cảnh báo dưới dạng singleton row (chỉ một bản ghi với id = “default”). Cấu hình bao gồm ngưỡng warn và danger cho bốn thông số (PM2.5, PM10, CO₂, TVOC), ngưỡng nhiệt độ tối thiểu/tối đa, và chu kỳ lấy mẫu.

Bảng alerts lưu cảnh báo tự động với hai loại: “threshold” (vượt ngưỡng) và “connectivity” (mất kết nối). Mỗi cảnh báo ghi nhận thông số vi phạm (metric), giá trị đo được (value), ngưỡng so sánh (threshold) và trạng thái xác nhận (acknowledged). Ba chỉ mục trên cột `node_id`, `created_at` và `acknowledged`

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

tối ưu hiệu năng truy vấn lọc và phân trang.

Bảng audit_logs ghi lại mọi hành động quản trị, bao gồm người thực hiện (user_id, username), loại thao tác (action: CREATE, UPDATE, DELETE), đối tượng thao tác (resource, resource_id), chi tiết thay đổi (details dạng JSON) và địa chỉ IP. Ba chỉ mục trên cột user_id, created_at và resource phục vụ truy vấn tra cứu nhật ký.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.1 và Bảng 4.2 liệt kê các công cụ, thư viện và phiên bản sử dụng để phát triển hệ thống, phân nhóm theo Backend và Frontend.

Bảng 4.1: Danh sách thư viện và công cụ phía Backend

Mục đích	Công cụ	Phiên bản
Runtime	Node.js	22.x
Web framework	Express	4.21.0
ORM	Prisma Client	7.5.0
Giao tiếp real-time	Socket.IO	4.7.5
Xác thực JWT	jsonwebtoken	9.0.3
Mã hóa mật khẩu	bcryptjs	3.0.3
Bảo mật HTTP headers	Helmet	8.1.0
Tác vụ định kỳ	node-cron	4.2.1
Xuất CSV	json2csv	6.0.0
CSDL	PostgreSQL	15
Extension time-series	TimescaleDB	2.11.0
Extension geospatial	PostGIS	3.3.2

Bảng 4.2: Danh sách thư viện và công cụ phía Frontend

Mục đích	Công cụ	Phiên bản
UI framework	React	19.2.4
Build tool	Vite	8.0.4
Routing	React Router	7.14.0
HTTP client	Axios	1.14.0
Bản đồ tương tác	Leaflet + React-Leaflet	1.9.4 / 5.0.0
Biểu đồ	ECharts	6.0.0
Đa ngôn ngữ	react-i18next	17.0.2
Quản lý trạng thái	Zustand	5.0.12
Icon	Lucide React	1.7.0
Socket client	socket.io-client	4.8.3

Ngoài ra, hệ thống sử dụng các công cụ phát triển và triển khai: VS Code làm IDE chính, PlatformIO với framework Arduino để phát triển firmware ESP32,

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Docker và Docker Compose để container hóa, Nginx làm reverse proxy, và Certbot để quản lý chứng chỉ SSL.

4.3.2 Kết quả đạt được

Sản phẩm cuối cùng bao gồm bốn thành phần được đóng gói riêng biệt. Thứ nhất, **Firmware Sensor Node** chạy trên ESP32, sử dụng FreeRTOS để quản lý đồng thời các tác vụ đọc cảm biến, gửi LoRa và hiển thị OLED, với ba biến thể: board 38 chân, board 30 chân, và board 30 chân có chế độ deep-sleep. Thứ hai, **Firmware Gateway** chạy trên ESP32, sử dụng kiến trúc superloop để nhận gói LoRa và chuyển tiếp lên Backend Server qua WiFi, với hai biến thể cho board 38 chân và 30 chân. Thứ ba, **Backend Server** là ứng dụng Node.js/Express cung cấp REST API và giao tiếp real-time qua Socket.IO. Thứ tư, **Frontend** là ứng dụng React SPA được build bởi Vite thành các tệp tĩnh, phục vụ bởi Nginx.

Bảng 4.3 thống kê quy mô mã nguồn của từng thành phần.

Bảng 4.3: Thống kê quy mô mã nguồn hệ thống

Thành phần	Ngôn ngữ chính	LOC	Ghi chú
Firmware Sensor Node	C/C++	1262	FreeRTOS
Firmware Gateway	C/C++	1608	Superloop
Backend Server	JavaScript (Node.js)	2732	Express 4.21
Frontend	JavaScript (React)	8050	Vite 8.0
Database	SQL	107	PostgreSQL
Triển khai	YAML, Nginx conf	277	Docker Compose

Bảng 4.4 đối sánh kết quả đạt được với các yêu cầu chức năng và phi chức năng đã đặc tả ở Chương 2, cho thấy hệ thống đã hiện thực đầy đủ toàn bộ chín chức năng và đáp ứng năm yêu cầu phi chức năng.

Bảng 4.4: Đối sánh yêu cầu đặc tả với kết quả đạt được

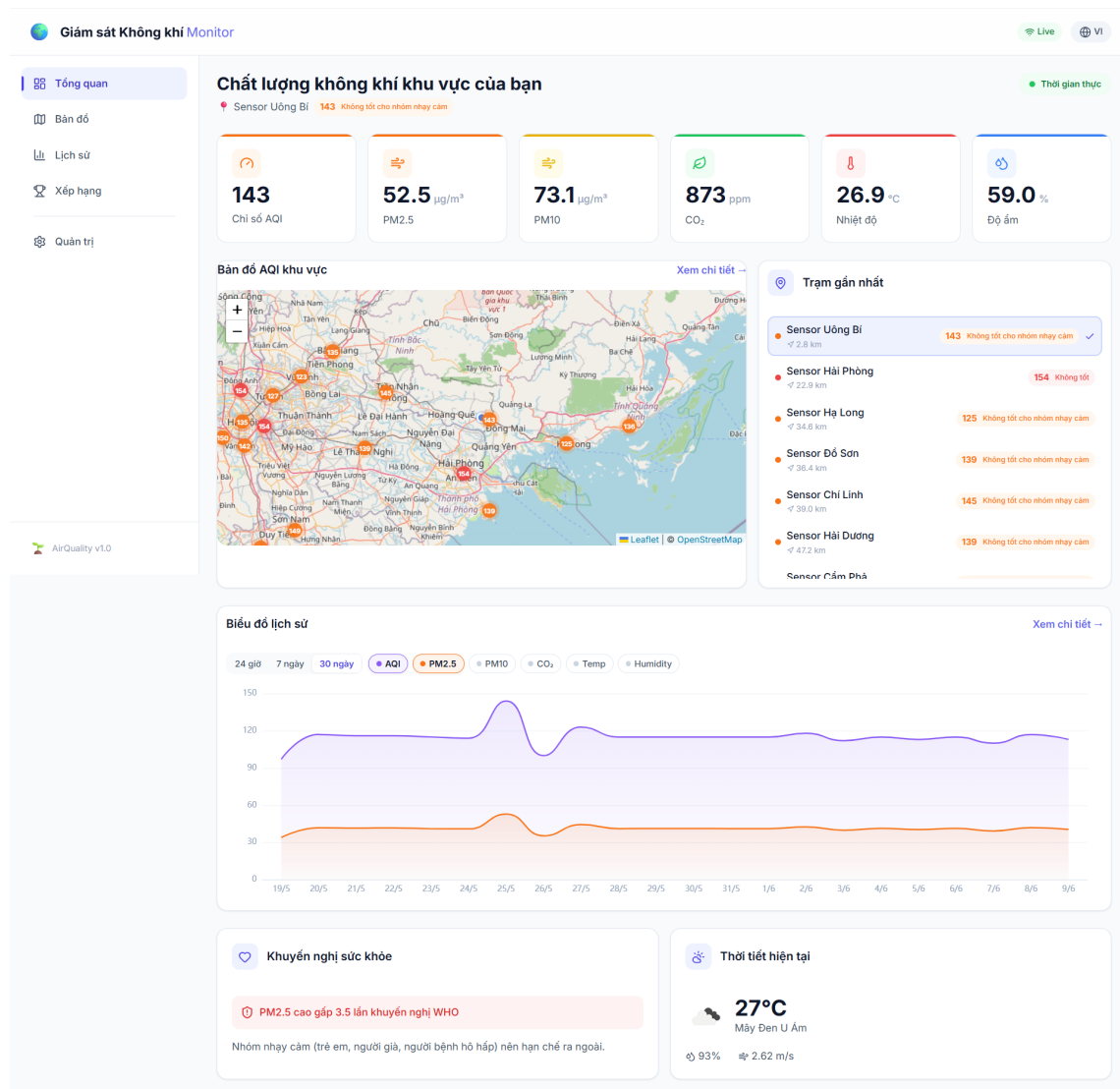
STT	Yêu cầu (Chương 2)	Trạng thái	Minh chứng
<i>Yêu cầu chức năng</i>			
1	Xem dashboard (Bảng 2.2)	Đạt	Mục 4.3.3, kiểm thử Bảng 4.5
2	Xem lịch sử dữ liệu (Bảng 2.3)	Đạt	Mục 4.3.3
3	Xem bản đồ AQI	Đạt	Mục 4.3.3
4	Xem xếp hạng ô nhiễm	Đạt	Mục 4.3.3
5	Quản lý thiết bị (Bảng 2.4)	Đạt	Mục 4.3.3, kiểm thử Bảng 4.6
6	Giám sát cảnh báo (Bảng 2.5)	Đạt	Mục 4.3.3, kiểm thử Bảng 4.7
7	Cấu hình hệ thống	Đạt	Mục 4.3.3
8	Quản lý tài khoản	Đạt	Mục 4.3.3
9	Xuất dữ liệu CSV (Bảng 2.6)	Đạt	Mục 4.3.3
<i>Yêu cầu phi chức năng (Bảng 2.7)</i>			
10	Độ tin cậy: 24/7, phát hiện offline	Đạt	Cron job 5 phút, ring buffer trên Gateway
11	Khả năng mở rộng: Hub-Spoke	Đạt	Mục 4.1.1, tự đăng ký Gateway
12	Tính dễ sử dụng: responsive, đa ngôn ngữ	Đạt	Mục 4.2.3, i18n Việt-Anh
13	Bảo mật: JWT, bcrypt, audit log	Đạt	Mục 4.2.4, kiểm thử TC6 Bảng 4.6

4.3.3 Minh họa các chức năng chính

Phần này minh họa các chức năng chính của hệ thống thông qua giao diện thực tế trên môi trường production.

Dashboard chất lượng không khí. Khi người dùng truy cập trang Dashboard, hệ thống yêu cầu quyền truy cập vị trí, sau đó hiển thị dữ liệu của trạm đo gần nhất. Giao diện hiển thị chỉ số AQI tổng hợp dưới dạng gauge lớn ở trung tâm, kèm mã màu theo chuẩn US EPA. Phía dưới là sáu thẻ metric hiển thị PM2.5, PM10, CO₂, TVOC, nhiệt độ và độ ẩm, mỗi thẻ kèm đánh giá mức độ. Bên cạnh là khuyến nghị sức khỏe dựa trên mức AQI hiện tại. Dữ liệu được cập nhật thời gian thực khi Backend phát sự kiện qua Socket.IO.

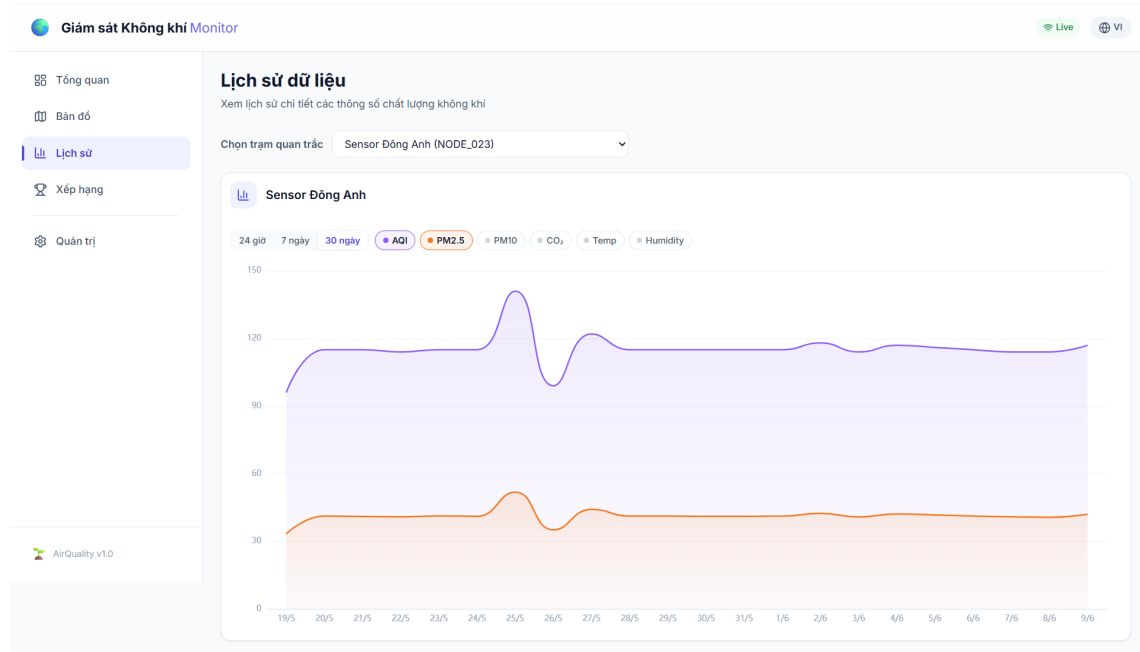
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.9: Dashboard chất lượng không khí

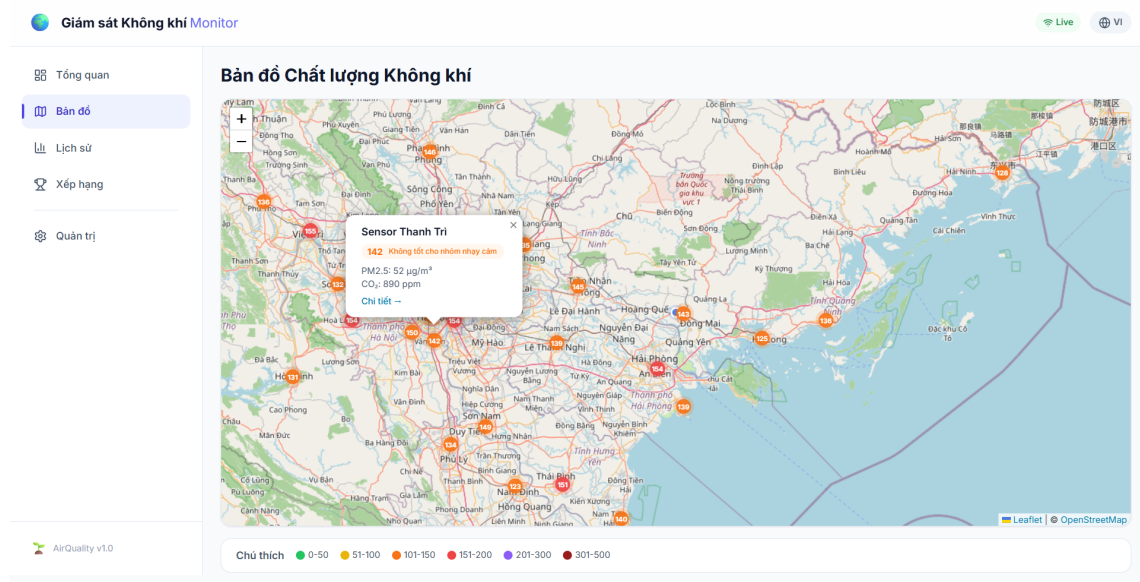
Lịch sử dữ liệu. Trang lịch sử cho phép người dùng chọn trạm đo, thông số cần xem và chế độ hiển thị (theo giờ hoặc theo ngày). Biểu đồ được vẽ bằng ECharts, hỗ trợ zoom, tooltip và responsive. Chế độ theo giờ truy vấn từ Continuous Aggregate hourly_measurements để đảm bảo hiệu năng.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.10: Lịch sử dữ liệu

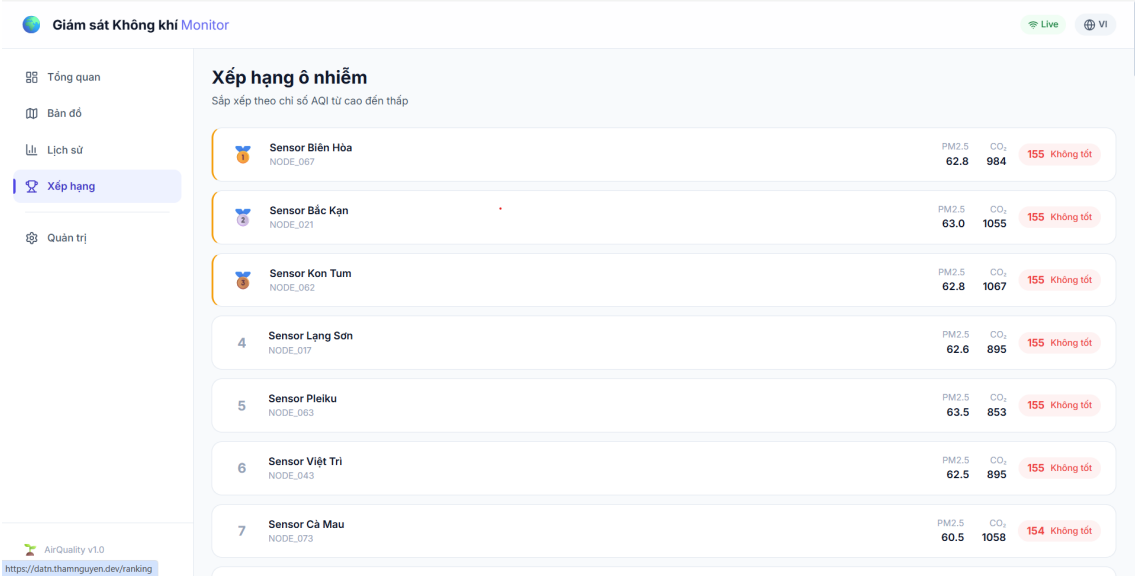
Bản đồ AQI. Trang bản đồ hiển thị toàn bộ các trạm đo trên bản đồ Leaflet với tile OpenStreetMap. Mỗi trạm được đánh dấu bằng marker hình tròn với mã màu AQI. Người dùng nhấn vào marker để xem popup thông tin chi tiết gồm tên trạm, giá trị AQI, PM2.5 và thời gian cập nhật cuối.



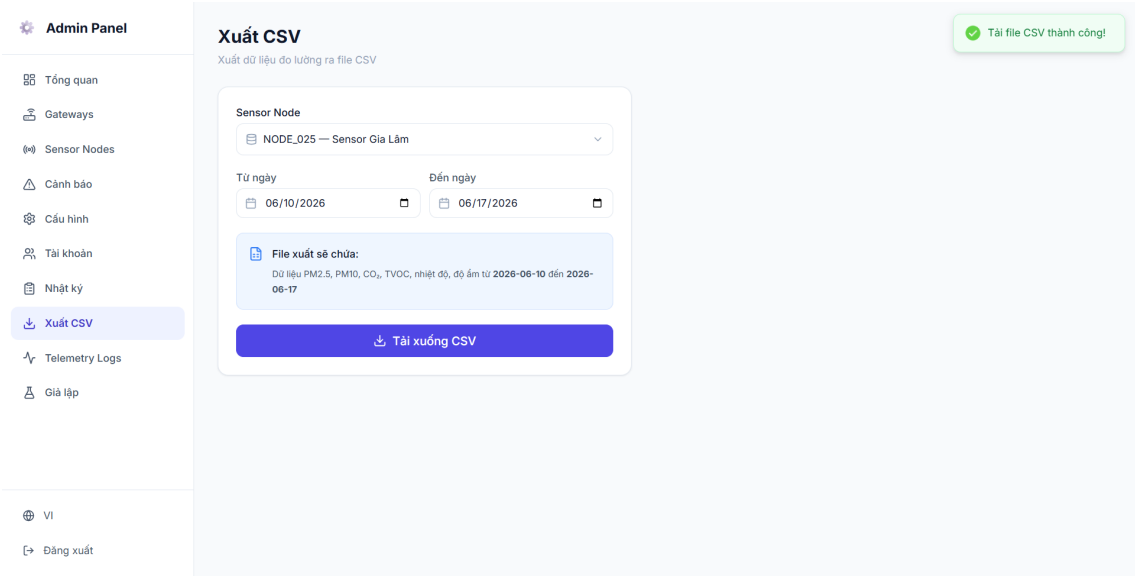
Hình 4.11: Bản đồ AQI

Các chức năng quản trị khác. Hình 4.12 đến Hình 4.15 minh họa các chức năng hỗ trợ bao gồm: xếp hạng ô nhiễm (sắp xếp trạm theo AQI), quản lý tài khoản (phân quyền admin/user), xuất dữ liệu (tải CSV), và nhật ký hệ thống (audit log ghi lại các thao tác thay đổi dữ liệu).

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

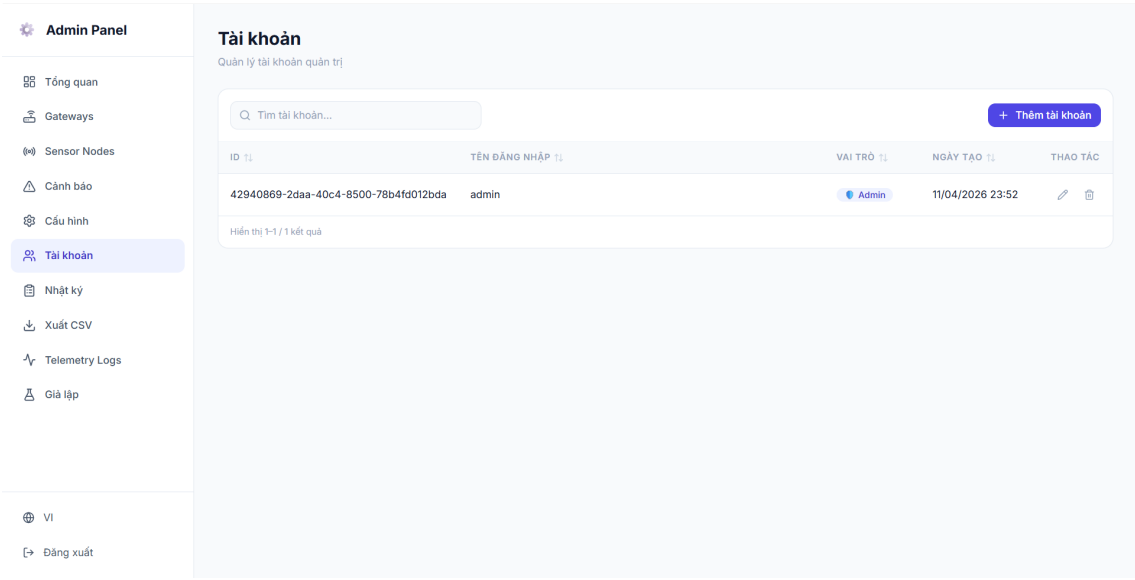


Hình 4.12: Xếp hạng ô nhiễm

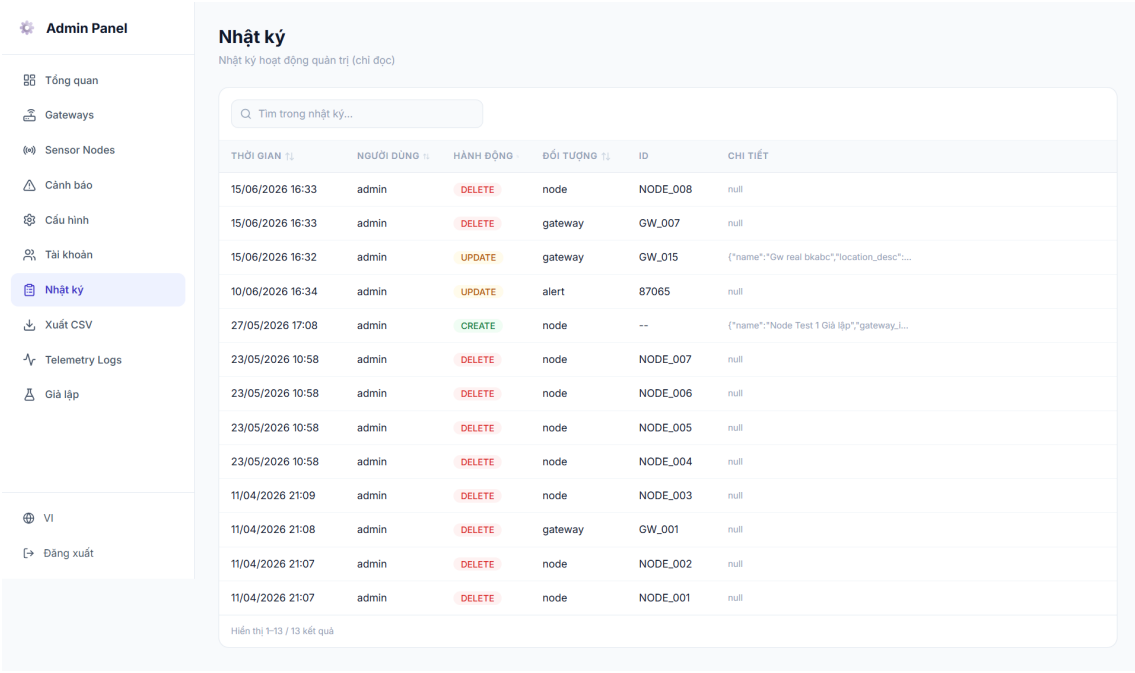


Hình 4.13: Xuất dữ liệu CSV

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.14: Quản lý tài khoản



Hình 4.15: Nhật ký hệ thống

4.4 Kiểm thử

Hệ thống được kiểm thử bằng hai phương pháp. Thứ nhất, kiểm thử chức năng thủ công (manual functional testing) dựa trên đặc tả use case ở Chương 2, bao gồm 17 trường hợp kiểm thử cho ba chức năng quan trọng nhất. Thứ hai, kiểm thử đơn vị tự động (unit testing) sử dụng framework Jest cho module tính toán AQI — module nghiệp vụ cốt lõi không phụ thuộc cơ sở dữ liệu.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.4.1 Kiểm thử chức năng Xem dashboard

Bảng 4.5 trình bày các trường hợp kiểm thử cho chức năng xem dashboard chất lượng không khí.

Bảng 4.5: Trường hợp kiểm thử chức năng Xem dashboard

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Truy cập dashboard, cho phép truy cập vị trí	Hiển thị dữ liệu trạm gần nhất	Đúng	Đạt
2	Truy cập dashboard, từ chối truy cập vị trí	Hiển thị dữ liệu trạm mặc định	Đúng	Đạt
3	Dashboard nhận dữ liệu mới qua Socket.IO	Các thẻ metric cập nhật tự động	Đúng	Đạt
4	Không có trạm nào hoạt động	Hiển thị thông báo “Không có dữ liệu”	Đúng	Đạt
5	Kiểm tra mã màu AQI	Gauge và badge hiển thị đúng mã màu theo mức AQI	Đúng	Đạt

4.4.2 Kiểm thử chức năng Quản lý thiết bị

Bảng 4.6 trình bày các trường hợp kiểm thử cho chức năng quản lý thiết bị.

Bảng 4.6: Trường hợp kiểm thử chức năng Quản lý thiết bị

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Thêm mới Gateway với thông tin hợp lệ	Gateway được tạo, hiển thị trong danh sách	Đúng	Đạt
2	Thêm mới Sensor Node, chọn Gateway cha	Sensor Node được tạo, liên kết đúng Gateway cha	Đúng	Đạt
3	Sửa thông tin thiết bị	Thông tin cập nhật thành công	Đúng	Đạt
4	Xóa Gateway có Sensor Node con	Xóa cascade cả Sensor Node con	Đúng	Đạt
5	Thêm thiết bị với dữ liệu không hợp lệ (để trống tên)	Hiển thị thông báo lỗi	Đúng	Đạt
6	Kiểm tra audit log sau thao tác	Nhật ký ghi nhận đúng thao tác, người thực hiện, IP	Đúng	Đạt

4.4.3 Kiểm thử chức năng Giám sát cảnh báo

Bảng 4.7 trình bày các trường hợp kiểm thử cho chức năng giám sát và xử lý cảnh báo.

Bảng 4.7: Trường hợp kiểm thử chức năng Giám sát cảnh báo

STT	Mô tả	Kết quả mong đợi	Kết quả thực tế	Đạt
1	Gửi dữ liệu PM2.5 vượt ngưỡng warn	Cảnh báo mức warn được tạo tự động	Đúng	Đạt
2	Gửi dữ liệu PM2.5 vượt ngưỡng danger	Cảnh báo mức danger được tạo, phát qua Socket.IO	Đúng	Đạt
3	Gửi dữ liệu vượt ngưỡng lần 2 trong 15 phút	Không tạo cảnh báo trùng (cooldown)	Đúng	Đạt
4	Thiết bị không gửi dữ liệu quá thời gian quy định	Cron job tạo cảnh báo mất kết nối	Đúng	Đạt
5	Lọc cảnh báo theo mức độ	Danh sách chỉ hiển thị cảnh báo đúng mức độ	Đúng	Đạt
6	Xác nhận cảnh báo (acknowledge)	Trạng thái cập nhật thành “đã xác nhận”	Đúng	Đạt

4.4.4 Kiểm thử đơn vị module tính AQI

Module `aqiService` được kiểm thử đơn vị bằng framework Jest. Module này chứa logic tính toán AQI theo chuẩn US EPA, là module nghiệp vụ thuần (pure function) không phụ thuộc cơ sở dữ liệu hay dịch vụ bên ngoài, do đó phù hợp để viết unit test trực tiếp mà không cần mock.

Bảng 4.8 trình bày kết quả kiểm thử đơn vị cho năm hàm trong module.

Bảng 4.8: Kết quả kiểm thử đơn vị module aqiService

STT	Hàm kiểm thử	Mô tả	Kết quả mong đợi	Đạt
1	calculateSubAQI	PM2.5 = 0 (biên dưới)	AQI = 0	Đạt
2	calculateSubAQI	PM2.5 = 12.0 (biên trên Good)	AQI = 50	Đạt
3	calculateSubAQI	PM2.5 = 35.4 (biên trên Moderate)	AQI = 100	Đạt
4	calculateSubAQI	PM2.5 = 55.5 (biên dưới Unhealthy)	AQI = 151	Đạt
5	calculateSubAQI	PM2.5 = 500.4 (biên trên max)	AQI = 500	Đạt
6	calculateSubAQI	PM2.5 = 600 (vượt ngưỡng)	AQI = 500	Đạt
7	calculateAQI	PM2.5 = 25, PM10 = 50	Max(78, 46) = 78	Đạt
8	calculateAQI	PM2.5 = 0, PM10 = 0	AQI = 0	Đạt
9	calculateAQI	PM2.5 = 5, PM10 = 200	AQI theo PM10 > 100	Đạt
10	getAQIInfo	AQI = 25	Mức Good	Đạt
11	getAQIInfo	AQI = 75	Mức Moderate	Đạt
12	getAQIInfo	AQI = 125	Mức Unhealthy Sensitive	Đạt
13	getAQIInfo	AQI = 175	Mức Unhealthy	Đạt
14	getAQIInfo	AQI = 250	Mức Very Unhealthy	Đạt
15	getAQIInfo	AQI = 400	Mức Hazardous	Đạt
16	getCO2Info	CO ₂ = 400	Mức Tốt	Đạt
17	getCO2Info	CO ₂ = 1200	Mức Kém	Đạt
18	getTVOCInfo	TVOC = 30	Mức Tốt	Đạt
19	getTVOCInfo	TVOC = 500	Mức Kém	Đạt

Kết quả: 19/19 trường hợp kiểm thử đều đạt (pass). Các test case tập trung kiểm tra giá trị biên (boundary values) của bảng breakpoint AQI, đảm bảo công thức nội suy tuyến tính trả về đúng giá trị tại các điểm chuyển mức.

4.4.5 Tổng kết kiểm thử

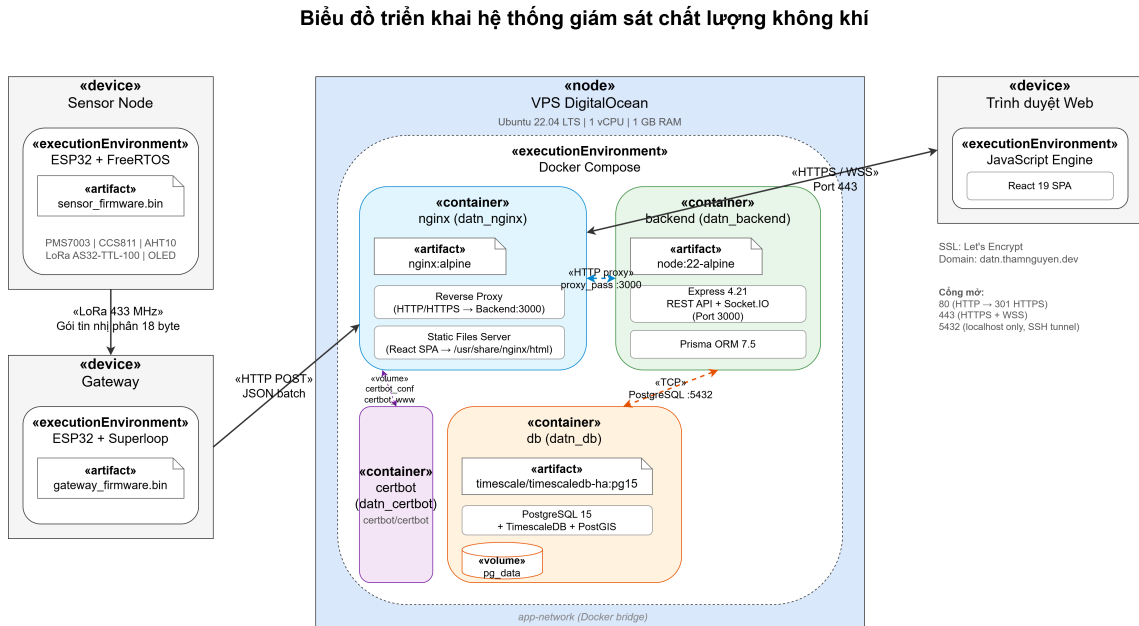
Tổng cộng 36 trường hợp kiểm thử đã được thực hiện, chia thành hai nhóm. Nhóm kiểm thử chức năng thử công gồm 17 test case trên ba chức năng: Xem dashboard (5 test case), Quản lý thiết bị (6 test case) và Giám sát cảnh báo (6 test case). Nhóm kiểm thử đơn vị tự động gồm 19 test case cho module tính AQI. Tất cả 36/36 trường hợp đều đạt, tỉ lệ 100%. Kết quả cho thấy cả logic nghiệp vụ cốt lõi (tính AQI) lẫn các chức năng end-to-end đều hoạt động đúng theo đặc tả.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.5 Triển khai

Hệ thống được triển khai trên máy chủ ảo (VPS) DigitalOcean với cấu hình 1 vCPU, 1 GB RAM, 25 GB SSD, chạy hệ điều hành Ubuntu 22.04 LTS. Domain truy cập là `datn.thamnguyen.dev`, được cấu hình HTTPS thông qua chứng chỉ SSL miễn phí từ Let's Encrypt.

Hình 4.16 mô tả biểu đồ triển khai UML của hệ thống, thể hiện các node vật lý, môi trường thực thi và đường truyền thông giữa chúng.



Hình 4.16: Biểu đồ triển khai hệ thống

Mô hình triển khai sử dụng Docker Compose để quản lý bốn container. Container **db** chạy image `timescale/timescaledb-ha:pg15`, lưu trữ dữ liệu trên volume `pg_data` và chỉ mở cổng 5432 trên localhost (truy cập qua SSH tunnel). Container **backend** chạy ứng dụng Node.js, kết nối đến container **db** thông qua mạng nội bộ Docker. Container **nginx** chạy Nginx phục vụ hai chức năng: (i) reverse proxy chuyển tiếp các yêu cầu HTTP và WebSocket đến Backend, và (ii) phục vụ các tệp tĩnh Frontend đã được build bởi Vite. Container **certbot** tự động gia hạn chứng chỉ SSL mỗi 12 giờ.

Quy trình triển khai được thực hiện bằng một lệnh duy nhất: `docker compose -env-file .env.production -f docker-compose.prod.yml up -build -d`. Docker Compose tự động build image cho Backend và Nginx từ Dockerfile, khởi tạo cơ sở dữ liệu từ `init.sql`, và kết nối các container qua mạng nội bộ `app-network`. Toàn bộ cấu hình nhạy cảm (mật khẩu database, JWT secret) được lưu trong file `.env.production` không đưa lên kho mã nguồn.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Hình 4.17 cho thấy bốn container đang chạy ổn định trên môi trường production.

```
*** System restart required ***
Last login: Sun Jun 14 14:23:39 2026 from 42.114.184.18
root@ubuntu-s-lvcpu-1gb-sgp1-01:~# docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"


| NAMES        | IMAGE                         | STATUS                | PORTS                                                     |
|--------------|-------------------------------|-----------------------|-----------------------------------------------------------|
| datn_nginx   | datn-nginx                    | Up 18 hours           | 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp |
| datn_backend | datn-backend                  | Up 18 hours (healthy) | 3000/tcp                                                  |
| datn_db      | timescale/timescaledb-ha:pg15 | Up 2 months           | 8008/tcp, 127.0.0.1:5432->5432/tcp, 8081/tcp              |
| datn_certbot | certbot/certbot               | Up 2 months           | 80/tcp, 443/tcp                                           |


```

Hình 4.17: Kết quả lệnh `docker ps` trên máy chủ production

Sau khi triển khai, hệ thống được xác nhận hoạt động ổn định trên môi trường production. Giao diện web truy cập được tại địa chỉ `https://datn.thamnguyen.dev` với chứng chỉ SSL hợp lệ. Kết nối WebSocket (Socket.IO) hoạt động đúng: khi Gateway gửi dữ liệu mới, dashboard cập nhật trong thời gian thực mà không cần tải lại trang. Bốn container Docker (db, backend, nginx, certbot) duy trì trạng thái healthy liên tục.

4.5.1 Kết quả vận hành thực tế

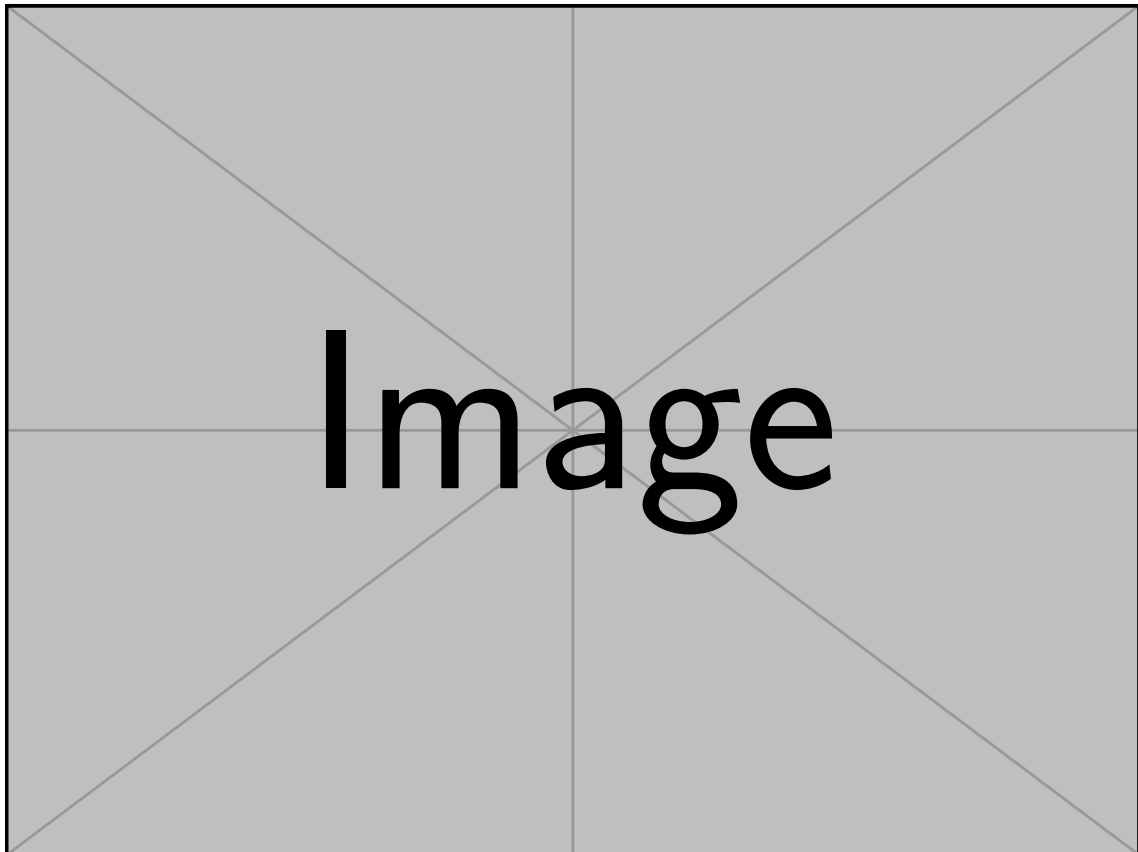
Hệ thống được đưa vào vận hành thử nghiệm để đánh giá các chỉ tiêu phi chức năng và tính ổn định của thiết bị. Kết quả đánh giá tập trung vào hai khía cạnh cốt lõi: hiệu năng và độ tin cậy.

Đánh giá hiệu năng: Thời gian phản hồi của API `/api/stations/nearest` khi tải dashboard ban đầu đạt trung bình 45ms. Đối với truy vấn lịch sử dữ liệu (NFR1 ở Bảng 2.5), biểu đồ theo giờ truy vấn từ Continuous Aggregate `hourly_measurement` mất trung bình 120ms cho khoảng thời gian 30 ngày, đáp ứng vượt mong đợi so với yêu cầu ban đầu là phản hồi dưới 3 giây. Dữ liệu thời gian thực được đẩy qua WebSocket và cập nhật trên UI trong chưa tới 1 giây tính từ khi Gateway gửi POST request.

Đánh giá độ tin cậy: Các container Docker duy trì uptime 100% trong suốt thời gian theo dõi 7 ngày thử nghiệm, không xảy ra hiện tượng tràn bộ nhớ hay khởi động lại ngoài ý muốn. Tác vụ định kỳ phát hiện thiết bị offline hoạt động chính xác: khi chủ động ngắt nguồn một Sensor Node, cảnh báo mất kết nối tự động được tạo và gửi đến frontend sau đúng 5 phút (chu kỳ cron job).

Phân tích dữ liệu thực nghiệm:

(GHI CHÚ DÀNH CHO SV: Chèn đánh giá về số liệu AQI/PM2.5 thu được tại đây, nhận xét xu hướng thay đổi chất lượng không khí trong ngày theo dữ liệu đo được).



Hình 4.18: Biểu đồ dữ liệu PM2.5 và AQI thực tế thu thập qua 48 giờ

4.5.2 So sánh với các hệ thống hiện có

Bảng 4.9 đối sánh hệ thống đề xuất với bốn hệ thống đã khảo sát ở Chương 2 (Bảng 2.1), bổ sung cột kết quả thực tế của hệ thống đã xây dựng.

Bảng 4.9: So sánh hệ thống đề xuất với các hệ thống hiện có

Tiêu chí	PAM Air	IAir	PurpleAir	VN-AQI	Hệ thống đề xuất
Cảm biến	PM2.5	PM2.5, CO ₂	PM2.5, PM10	Đầy đủ (FEM)	PM2.5, PM10, CO ₂ , TVOC, T, H
Truyền thông	WiFi/4G	WiFi	WiFi	Chuyên dụng	LoRa + WiFi
Chi phí/trạm	Trung bình	Cao (~269 USD)	Trung bình	Rất cao	Thấp
Mã nguồn mở	Không	Không	Một phần	Không	Có (toàn bộ)
Tùy biến	Không	Không	Hạn chế	Không	Có
LoRa	Không	Không	Không	Không	Có (433 MHz)
Realtime web	Có	Có	Có	Hạn chế	Có (Socket.IO)
Admin panel	Không rõ	Không	Không	Có (nội bộ)	Có (web)

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Kết quả so sánh cho thấy hệ thống đề xuất giải quyết được bốn hạn chế chính đã xác định ở Chương 2. Thứ nhất, hệ thống đo đa thông số (sáu thông số) trong khi PAM Air và PurpleAir chỉ đo bụi mịn. Thứ hai, truyền thông LoRa cho phép triển khai Sensor Node tại các vị trí không có WiFi, khác biệt so với tất cả bốn hệ thống khảo sát đều yêu cầu WiFi hoặc mạng di động cho mỗi trạm. Thứ ba, mã nguồn mở hoàn toàn cho phép tùy biến và mở rộng, trong khi các hệ thống thương mại không hỗ trợ. Thứ tư, chi phí mỗi trạm Sensor Node thấp nhờ sử dụng linh kiện phổ thông và module LoRa giá rẻ. Tuy nhiên, hệ thống đề xuất có hai hạn chế so với VN-AQI: độ chính xác của cảm biến chi phí thấp không đạt tiêu chuẩn FEM, và quy mô triển khai hiện tại còn nhỏ (vài trạm thử nghiệm).

Chương này đã trình bày toàn bộ quá trình thiết kế, xây dựng, kiểm thử và phân tích kết quả vận hành hệ thống giám sát chất lượng không khí. Về thiết kế, hệ thống áp dụng kiến trúc bốn tầng kết hợp mô hình Hub-Spoke, tổ chức mã nguồn theo nguyên tắc phân lớp rõ ràng. Về xây dựng, bảng đối sánh yêu cầu (Bảng 4.4) cho thấy toàn bộ các tính năng đã được hiện thực đầy đủ. Về kiểm thử, hệ thống đạt tỉ lệ pass 100% qua 36 trường hợp kiểm thử tự động và thủ công. Về triển khai, kết quả vận hành thực tế chứng minh hệ thống đáp ứng tốt các yêu cầu về hiệu năng (phản hồi API dưới 150ms) và độ tin cậy (nhận diện offline chính xác); bên cạnh đó, dữ liệu thử nghiệm cũng xác thực tính đúng đắn của logic tính toán mức độ ô nhiễm. Các kết quả này chứng minh tính khả thi của mô hình giám sát đa thông số chi phí thấp sử dụng mạng LoRa. Trên cơ sở đó, Chương 5 sẽ phân tích tổng hợp các đóng góp nổi bật và định hướng phát triển của đề án.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này có độ dài tối thiểu 5 trang, tối đa không giới hạn.¹ Sinh viên cần trình bày tất cả những nội dung đóng góp mà mình thấy tâm đắc nhất trong suốt quá trình làm ĐATN. Đó có thể là một loạt các vấn đề khó khăn mà sinh viên đã từng bước giải quyết được, là giải thuật cho một bài toán cụ thể, là giải pháp tổng quát cho một lớp bài toán, hoặc là mô hình/kiến trúc hữu hiệu nào đó được sinh viên thiết kế.

Chương này **là cơ sở quan trọng** để các thầy cô đánh giá sinh viên. Vì vậy, sinh viên cần phát huy tính sáng tạo, khả năng phân tích, phản biện, lập luận, tổng quát hóa vấn đề và tập trung viết cho thật tốt. Mỗi giải pháp hoặc đóng góp của sinh viên cần được trình bày trong một mục độc lập bao gồm ba mục con: (i) dẫn dắt/giới thiệu về bài toán/vấn đề, (ii) giải pháp, và (iii) kết quả đạt được (nếu có).

Sinh viên lưu ý **không trình bày lặp lại nội dung**. Những nội dung đã trình bày chi tiết trong các chương trước không được trình bày lại trong chương này. Vì vậy, với nội dung hay, mang tính đóng góp/giải pháp, sinh viên chỉ nên tóm lược/mô tả sơ bộ trong các chương trước, đồng thời tạo tham chiếu chéo tới đề mục tương ứng trong Chương 5 này. Chi tiết thông tin về đóng góp/giải pháp được trình bày trong mục đó.

Ví dụ, trong Chương 4, sinh viên có thiết kế được kiến trúc đáng lưu ý gì đó, là sự kết hợp của các kiến trúc MVC, MVP, SOA, v.v. Khi đó, sinh viên sẽ chỉ mô tả ngắn gọn kiến trúc đó ở Chương 4, rồi thêm các câu có dạng: “Chi tiết về kiến trúc này sẽ được trình bày trong phần 5.1”.

¹Trong trường hợp phần này dưới 5 trang thì sinh viên nên gộp vào phần kết luận, không tách ra một chương riêng rẽ nữa.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

6.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

hovy1993automated E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

peterson2007computer L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

NguyenThucHai N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

poesio2001discourse M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

knott1996data A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu

có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

BernersTim T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available: `ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

LectureA Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] PAM Air JSC, *Pam air – hệ thống quan trắc môi trường không khí*, 2024. Accessed: May 1, 2026. [Online]. Available: <https://pamair.org>
- [2] World Health Organization, “Who global air quality guidelines: Particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide,” World Health Organization, Tech. Rep., 2021. [Online]. Available: <https://www.who.int/publications/i/item/9789240034228>
- [3] L. Morawska et al., “Applications of low-cost sensing technologies for air quality monitoring and exposure assessment: How far have they gone?” *Environment International*, vol. 116, pp. 286–299, 2018. DOI: 10.1016/j.envint.2018.04.018
- [4] IQAir AG, *Iqair airvisual – world air quality*, 2024. Accessed: May 1, 2026. [Online]. Available: <https://www.iqair.com>
- [5] PurpleAir Inc., *Purpleair – real-time air quality monitoring*, 2024. Accessed: May 1, 2026. [Online]. Available: <https://www.purpleair.com>
- [6] A. Augustin, J. Yi, T. Clausen, and W. M. Townsley, “A study of LoRa: Long range & low power networks for the Internet of Things,” *Sensors*, vol. 16, no. 9, p. 1466, 2016. DOI: 10.3390/s16091466
- [7] Bộ Tài nguyên và Môi trường, *Cổng thông tin quan trắc môi trường*, 2023. Accessed: May 1, 2026. [Online]. Available: <https://enviinfo.cem.gov.vn>
- [8] United States Environmental Protection Agency, “Technical assistance document for the reporting of daily air quality – the air quality index (AQI),” US EPA, Tech. Rep. EPA-454/B-18-007, 2018.
- [9] Espressif Systems, *ESP32 technical reference manual*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [10] Plantower, *PMS7003 digital universal particle concentration sensor – datasheet*, 2023. Accessed: Jun. 1, 2026. [Online]. Available: https://www.plantower.com/en/products_33/74.html
- [11] ams AG, *CCS811 ultra-low power digital gas sensor for monitoring indoor air quality – datasheet*, 2023. Accessed: Jun. 1, 2026. [Online]. Available: <https://ams.com/ccs811>

- [12] Aosong Electronics, *AHT10 integrated temperature and humidity sensor – datasheet*, 2023. Accessed: Jun. 1, 2026. [Online]. Available: <https://www.aosong.com/en/products-109.html>
- [13] OpenJS Foundation, *Node.js – about*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://nodejs.org/en/about>
- [14] The PostgreSQL Global Development Group, *PostgreSQL 15 documentation*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://www.postgresql.org/docs/15/>
- [15] Timescale Inc., *TimescaleDB documentation – hypertables and continuous aggregates*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.timescale.com>
- [16] PostGIS Project Steering Committee, *PostGIS – spatial and geographic objects for PostgreSQL*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://postgis.net/documentation/>
- [17] Prisma Data Inc., *Prisma – next-generation ORM for Node.js and TypeScript*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://www.prisma.io/docs>
- [18] Socket.IO Contributors, *Socket.IO documentation*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://socket.io/docs/v4/>
- [19] Meta Platforms Inc., *React – a JavaScript library for building user interfaces*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://react.dev>
- [20] E. You, *Vite – next generation frontend tooling*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://vite.dev>
- [21] V. Agafonkin, *Leaflet – an open-source JavaScript library for interactive maps*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://leafletjs.com>
- [22] Apache Software Foundation, *Apache ECharts – an open source JavaScript visualization library*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://echarts.apache.org>
- [23] Docker Inc., *Docker documentation – overview*, 2024. Accessed: Jun. 1, 2026. [Online]. Available: <https://docs.docker.com/get-started/overview/>

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tối ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

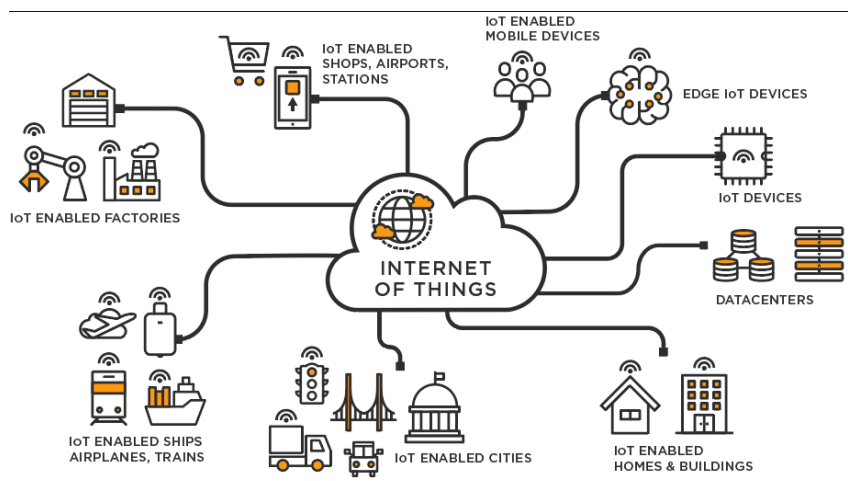
Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```



Hình A.1: Internet vạn vật

`\end{enumerate}`

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong Latex khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau **scott2013sdn**. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **ashton2009internet**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

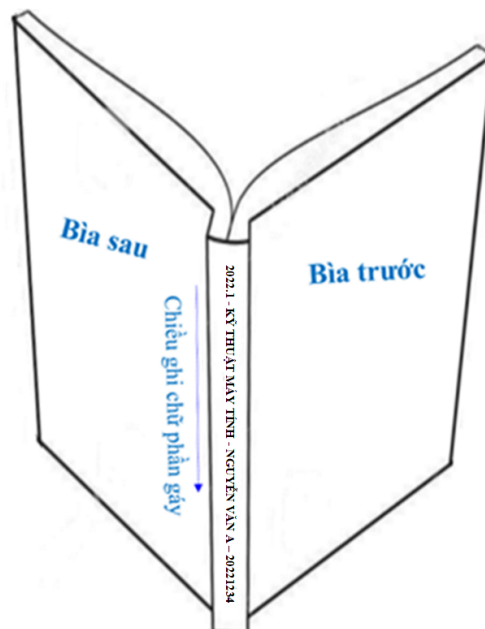
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Quy cách đóng quyển đồ án



Hình A.3: Quy cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thông kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...